



School of Future Tech

Case Study Report

on

TownRate – Local Business Review Platform

by

Aditya Sunil Chouksey

150096725070

Index

1. Introduction to the Case Study.
2. Problem Statement / Case Background (Abstract).
3. Problem Statement / Case Study Design.
4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study.
5. Problem Statement / Case Study Implementation Details and Snapshots.
6. Problem Statement / Case Study Results and Conclusion.
7. References

Appendix A – Complete Source Code

1. Introduction to the Case Study

People increasingly decide where to eat, shop or work out by reading what others in their town say about a place. Small local businesses, in turn, need a simple way to be found and to answer feedback. This case study builds the backend for **TownRate**, a Local Business Review Platform, using **Node.js**, **Express.js**, **MongoDB** and **Firebase**.

TownRate is a REST API. Business owners create business profiles with photos, anyone can search for businesses by keyword, category or location (including a "near me" search), and registered users rate and review businesses. Owners can reply to reviews. The API is protected with JSON Web Tokens (JWT), supports Firebase Authentication, and stores business photos in Firebase Storage.

The case study shows how a real-world backend is split into routes, controllers, models and middleware; how data is modelled in MongoDB with Mongoose; how authentication and role-based access control work; and how the service is documented, tested and deployed on Render.

1.1 Deliverables and where they are covered

The table maps every requirement in the problem statement (PS) to the section of this report that covers it.

Table 1: Problem-statement requirements mapped to report sections

Requirement (quoted from the PS)	Status	Section
"Business owners can create business profiles."	Done	3.4, 5.2
"Users can search businesses by category/location."	Done	3.4, 4.1, 5.2
"Rate and review businesses."	Done	3.3, 3.6, 5.2
"Firebase Storage for business photos."	Done	3.1, 4.1, 5.2
"Node.js + Express.js REST API with modular routes and controllers."	Done	3.2, 4.2
"MongoDB with Mongoose ODM for businesses, users, and reviews."	Done	3.3
"JWT-based authentication + Firebase Auth."	Done	3.5, 3.7, 4.1
API endpoints: auth, businesses, reviews, categories, search	Done	3.4
"Middleware: JWT authentication, file upload middleware, validation middleware."	Done	3.1, 4.1, 5.2
Optional: "Geo-location 'near me' search API."	Done	4.1, 5.2
Optional: "Business owner response-to-review API."	Done	3.4, 5.3
"REST API with all CRUD operations."	Done	3.4
"API documentation (Swagger/Postman collection)."	Both provided	5.4

Requirement (quoted from the PS)	Status	Section
"GitHub repository with README and setup instructions."	Done	5.1, 7
Bonus: "Deployment on Render/Heroku."	Render	5.5
Example user flow: search → review → view reviews	Done	5.3

2. Problem Statement / Case Background (Abstract)

Background

Communities want to discover and review local businesses, but general-purpose map and listing services are crowded and give small shops little control over their profile. A focused platform needs a backend that keeps business profiles, photos, ratings and reviews consistent, lets people search in natural ways ("cafe", "gyms in Pune", "what is near me"), and stops abuse such as one person posting many reviews or an owner rating their own shop.

The problem statement (Case Study 95) asks for the backend API of such a platform, called "TownRate", using Node.js, Express.js, MongoDB with Mongoose, JWT authentication, Firebase Auth and Firebase Storage, with modular routes and controllers and middleware for authentication, file uploads and validation.

Abstract

This case study presents the design and implementation of the TownRate REST API. The service is written in Node.js with Express 5.2.1 and stores users, businesses, reviews and categories in MongoDB through Mongoose 9.10.2. Users register and log in with email and password (passwords hashed with bcrypt) and receive a signed JWT; alternatively, a Firebase ID token can be exchanged for the same JWT. Three roles – user, owner and admin – control what each person may do, and ownership checks make sure owners can only change their own businesses.

Businesses store a GeoJSON location with a 2dsphere index, which powers a "near me" search sorted by distance. Keyword search matches names, descriptions, tags, cities and category names. Each review updates a cached average rating through a MongoDB aggregation, and a unique index allows only one review per user per business. Photos are uploaded with multer and stored in Firebase Storage. Every input is validated with express-validator, and a central error handler returns consistent JSON errors.

The API is documented with an OpenAPI 3 specification (Swagger UI) and a Postman collection of 41 requests, covered by an automated test suite of 41 tests, and deployed on Render with MongoDB Atlas.

3. Case Study Design

TownRate is designed as a layered REST service. Each layer has one job, so the code is easy to read, test and change.

3.1 System architecture

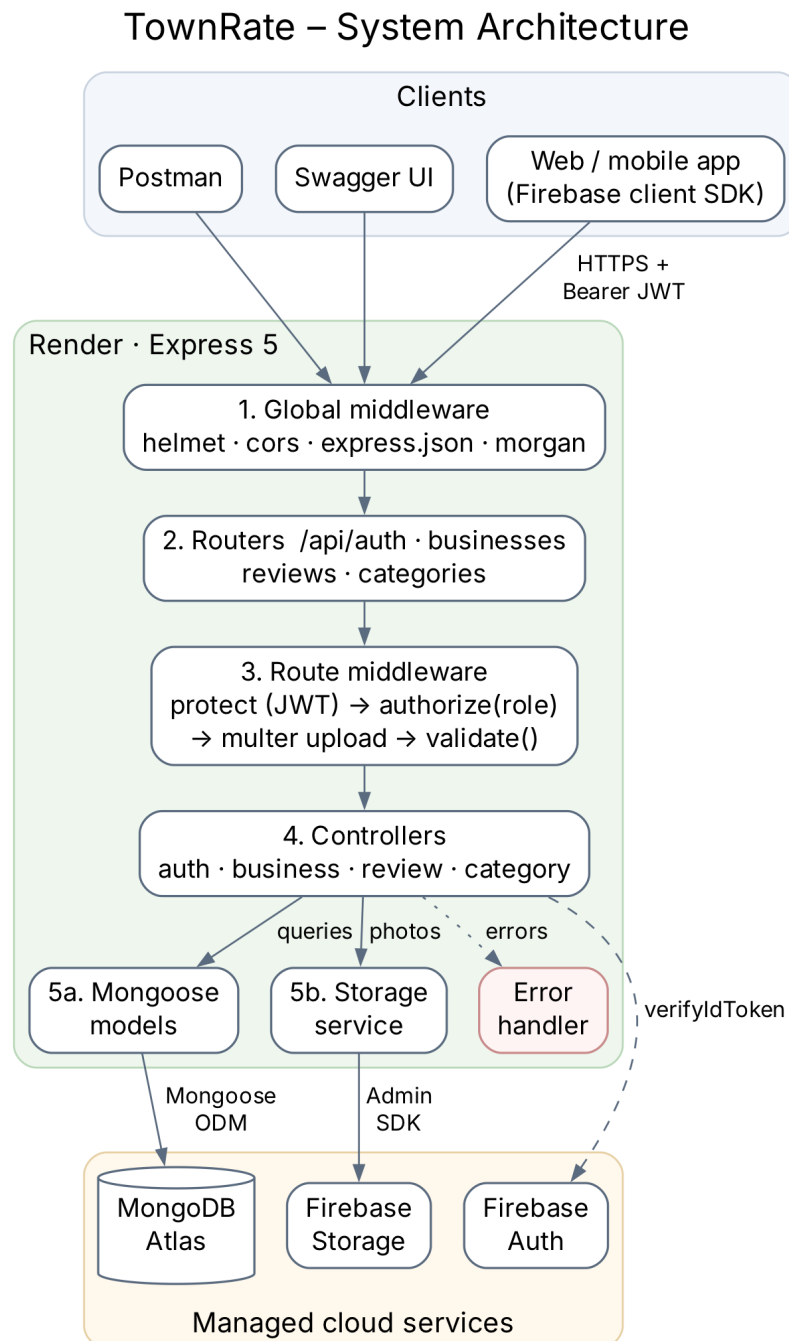


Figure 1: System architecture of TownRate

A request travels through the layers in order:

1. **Global middleware** – `helmet` sets security headers, `cors` allows browser clients from other origins, `express.json` parses JSON bodies and `morgan` logs each request.
2. **Routers** – one router per resource (`/api/auth`, `/api/businesses`, `/api/reviews`, `/api/categories`) decides which handler chain runs.
3. **Route middleware** – `protect` checks the JWT, `authorize` checks the role, `uploadPhotos` (multer) reads image files and `validate` checks the input. Any of them can stop the request early with an error.
4. **Controllers** – contain the business rules (for example "you cannot review your own business") and build the response.
5. **Models and services** – Mongoose models talk to MongoDB Atlas; the storage service sends photos to Firebase Storage through the Firebase Admin SDK.
6. **Error handler** – every error ends up in one function that turns it into a JSON response with the right HTTP status code.

3.2 Module structure

Table 2: Project folders and their responsibility

Folder / file	Responsibility
<code>src/server.js</code>	Connects to MongoDB, then starts the HTTP server (used locally and on Render)
<code>src/app.js</code>	Builds the Express app: global middleware, Swagger docs, routes, 404 and error handler
<code>src/config/</code>	Environment variables (<code>env.js</code>), MongoDB connection (<code>db.js</code>), Firebase Admin setup (<code>firebase.js</code>)
<code>src/models/</code>	Mongoose schemas: User, Business, Review, Category
<code>src/routes/</code>	One router per resource; wires URL + middleware + controller
<code>src/controllers/</code>	Request handlers with the business rules
<code>src/middleware/</code>	<code>auth.js</code> (JWT, roles), <code>upload.js</code> (multer), <code>validate.js</code> , <code>errorHandler.js</code>
<code>src/validators/</code>	express-validator rules for every endpoint
<code>src/services/storage.js</code>	Upload/delete photos in Firebase Storage (local disk fallback for development)
<code>src/utils/</code>	<code>ApiError</code> class; pagination, sorting, regex-escaping helpers
<code>src/docs/</code>	OpenAPI 3 specification and the Swagger UI page
<code>scripts/seed.js</code>	Creates categories and demo data
<code>tests/</code>	Unit tests and end-to-end API tests
<code>postman/</code>	Postman collection and a sample photo

3.3 Data model

TownRate – Data Model (MongoDB collections)

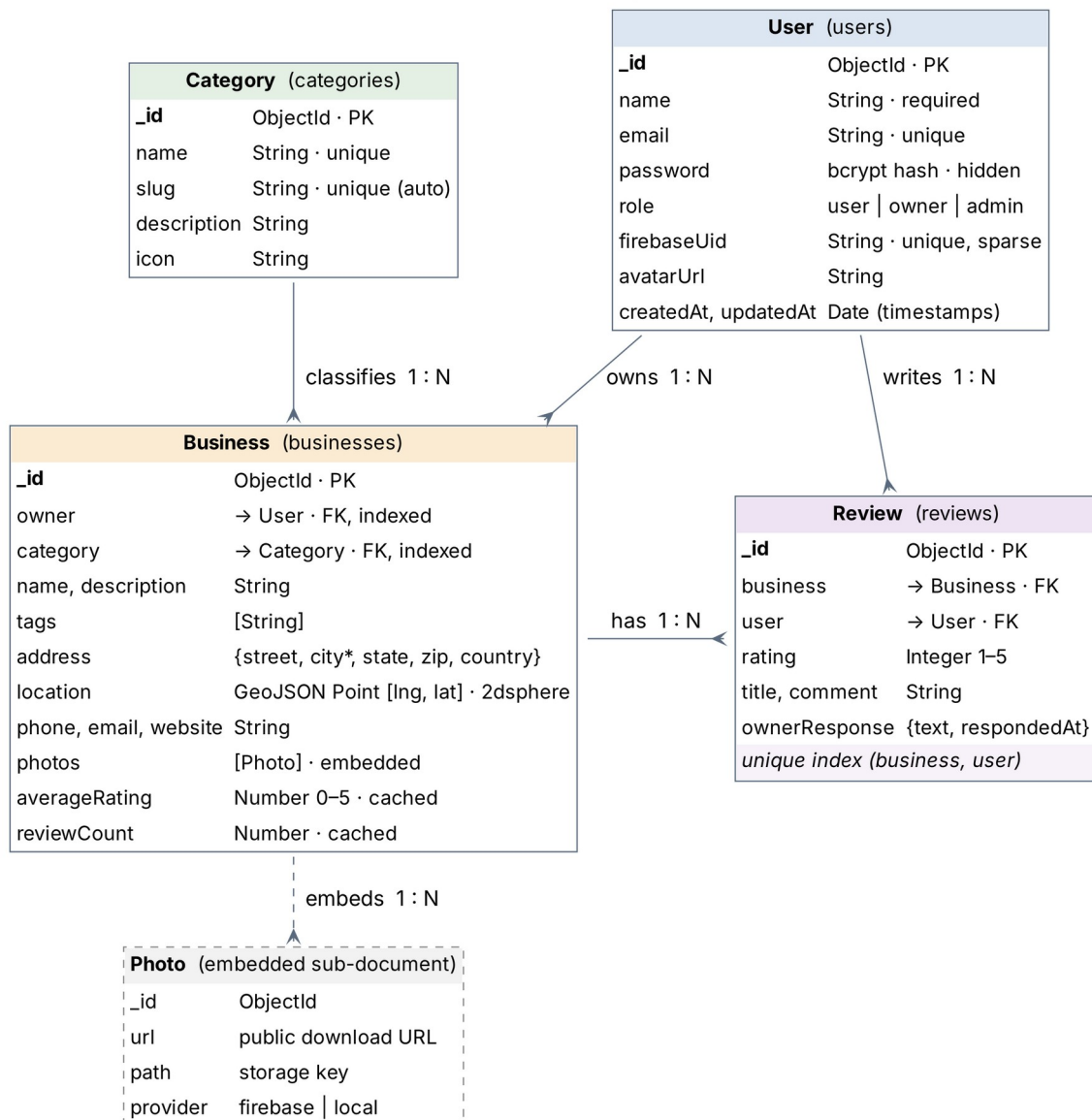


Figure 2: Collections, fields and relationships

MongoDB stores documents, not table rows, so the design chooses between **referencing** (store another document's `_id`) and **embedding** (store the data inside the parent document):

- **Referenced:** a business references its owner (User) and its Category; a review references its Business and its User. These are separate collections because they are queried on their own (for example "all reviews of a business", "my reviews").
- **Embedded:** photos live inside the business document. A photo is never needed without its business, so embedding saves a query.

- **Cached values:** `averageRating` and `reviewCount` are stored on the business so lists can be sorted and filtered by rating without reading every review. They are recalculated whenever a review is created, edited or deleted.

Table 3: Indexes and the query each one serves

Index	Type	Why
<code>users.email</code>	unique	Fast login lookup; no duplicate accounts
<code>users.firebaseUid</code>	unique, sparse	Link Firebase accounts; sparse because most users have none
<code>businesses.location</code>	2dsphere	Required for the "near me" (\$geoNear) search
<code>businesses.owner, .category, .address.city, .averageRating</code>	single field	Filters and sorting on the list endpoint
<code>reviews.(business, user)</code>	unique compound	At most one review per user per business, even with simultaneous requests
<code>categories.name, .slug</code>	unique	No duplicate categories; lookup by slug

3.4 API design

All endpoints live under `/api`, use JSON and follow REST conventions: nouns in the URL, the HTTP method says what happens. Successful responses look like `{ "success": true, "data": ... }` (lists also include `pagination`); errors look like `{ "success": false, "message": ..., "errors": [...] }`. The last column shows whether the endpoint is required by the PS, optional in the PS, or added to complete CRUD and usability.

Table 4: TownRate API endpoints

Method	Path	Access	Purpose	PS
POST	<code>/api/auth/register</code>	Public	Create an account (role user or owner) and return a JWT	Required
POST	<code>/api/auth/login</code>	Public	Check email + password, return a JWT	Required
POST	<code>/api/auth/firebase</code>	Public	Exchange a Firebase ID token for an app JWT	Added
GET	<code>/api/auth/me</code>	Logged in	Current user profile	Added
GET	<code>/api/businesses</code>	Public	List businesses; filter by category, city, minRating, owner; sort; paginate	Required
GET	<code>/api/businesses/search?keyword=cafe</code>	Public	Keyword search (name, description, tags, city, category)	Required

Method	Path	Access	Purpose	PS
GET	/api/businesses/near?lat=&lng=&radius=	Public	"Near me" geo search, nearest first with distanceKm	Optional (PS)
GET	/api/businesses/:id	Public	One business with category and owner	Required
POST	/api/businesses	Owner, admin	Create a business profile (JSON or multipart with photos)	Required
PUT	/api/businesses/:id	Its owner, admin	Partial update; new photos are appended	Required
DELETE	/api/businesses/:id	Its owner, admin	Delete business with its reviews and photos	Added
POST	/api/businesses/:id/photos	Its owner, admin	Upload photos to Firebase Storage	Added
DELETE	/api/businesses/:id/photos/:photoId	Its owner, admin	Delete one photo	Added
POST	/api/reviews	Logged in	Leave a review (one per user per business)	Required
GET	/api/reviews/business/:id	Public	All reviews for a business + star breakdown	Required
GET	/api/reviews/me	Logged in	Current user's reviews	Added
PUT	/api/reviews/:id	Author	Edit own review	Added
DELETE	/api/reviews/:id	Author, admin	Delete a review	Added
POST / PUT	/api/reviews/:id/response	Business owner, admin	Owner responds to a review	Optional (PS)
DELETE	/api/reviews/:id/response	Business owner, admin	Remove the owner response	Added
GET	/api/categories	Public	All categories with business counts	Required
POST	/api/categories	Admin	Create category	Added
PUT	/api/categories/:id	Admin	Update category	Added
DELETE	/api/categories/:id	Admin	Delete unused category	Added
GET	/api/health	Public	Health check: database and Firebase status	Added
GET	/api-docs	Public	Swagger UI (interactive documentation)	Added

3.5 Roles and permissions

Table 5: Who may do what

Action	User	Owner	Admin
Search, list and view businesses and reviews	Yes	Yes	Yes
Leave, edit or delete own review	Yes	Yes (not on own business)	Yes

Action	User	Owner	Admin
Create a business	No	Yes	Yes
Edit or delete a business, manage its photos	No	Only own	Any
Respond to a review	No	Only on own business	Any
Delete any review	No	No	Yes
Create, edit, delete categories	No	No	Yes

Self-registration can only choose `user` or `owner`; admin accounts are created directly in the database (or by the seed script). This prevents anyone from making themselves an administrator.

3.6 Request flow: leaving a review

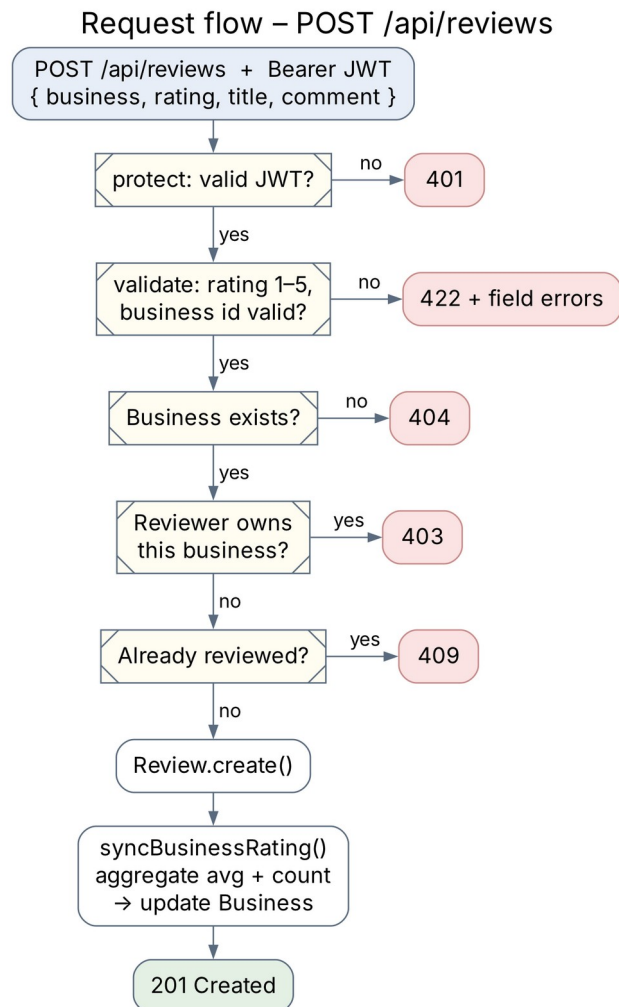


Figure 3: Checks performed for POST /api/reviews

The flow shows the "fail early" design: cheap checks (token, input format) run first, database checks after, and the write happens only when every rule passes. After the review is saved the business rating is recalculated, so the response already contains the new average.

3.7 Authentication design

Authentication – email/password JWT and Firebase token exchange

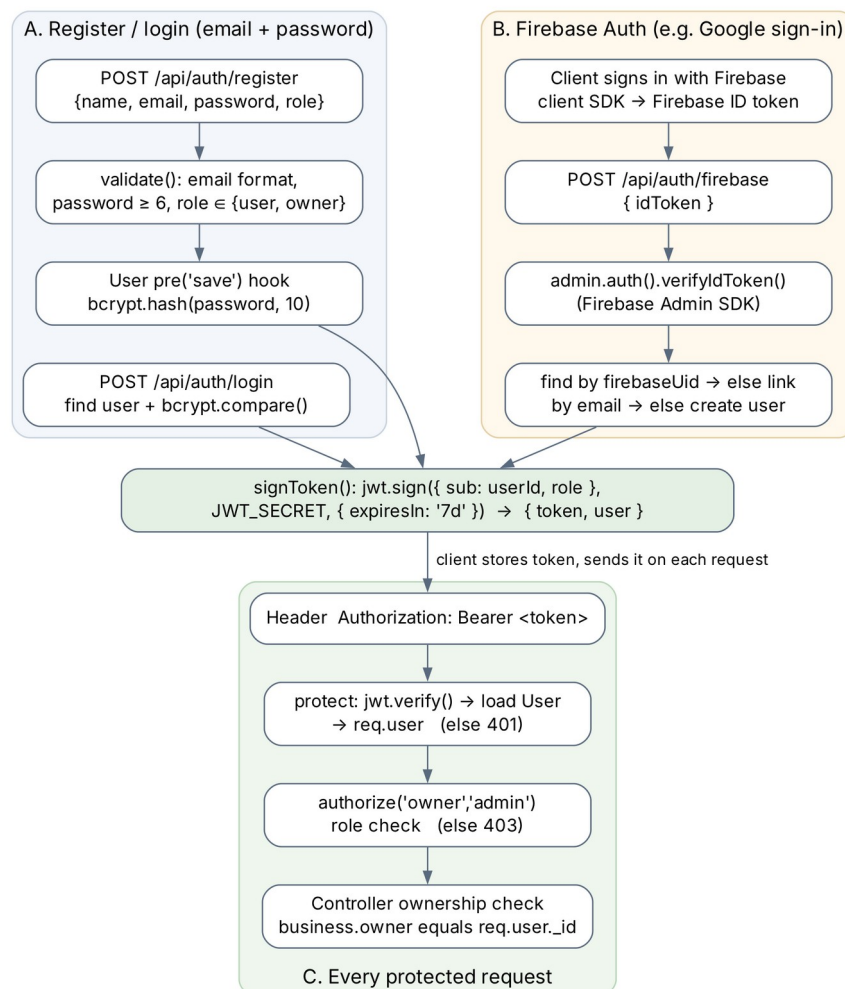


Figure 4: Email/password login, Firebase token exchange and protected requests

Both ways of signing in end with the same app JWT. A frontend that uses Firebase (for example Google sign-in) sends its Firebase ID token once to `/api/auth/firebase`; the server verifies it with the Firebase Admin SDK, finds or creates the matching user and returns an app JWT. From then on every protected route works the same way, whichever login method was used.

4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study

4.1 Key methods

1. **Password hashing (bcrypt).** Passwords are never stored. A Mongoose `pre('save')` hook hashes them with bcrypt (cost factor 10), which adds a random salt and is deliberately slow, so leaked hashes are hard to crack. Login uses `bcrypt.compare`.
2. **Token-based authentication (JWT, HS256).** After login the server signs `{ sub: userId, role }` with a secret key and an expiry of 7d. The client sends it as `Authorization: Bearer <token>`. The server only needs the secret to verify it, so no session is stored (stateless).
3. **Firestore token exchange.** `admin.auth().verifyIdToken()` checks a Firestore ID token's signature and expiry against Google's public keys. The user is then found by Firestore UID, linked by email, or created.
4. **Role-based access control and ownership checks.** `authorize(...roles)` rejects users without the right role (403). Controllers also compare `business.owner` with the logged-in user so owners can change only their own data.
5. **Input validation.** express-validator chains check types, ranges and formats (email, rating 1–5, coordinates, ObjectIds) and return HTTP 422 with an error per field before any database work happens.
6. **Keyword search.** The keyword is escaped (so characters like `.*` are treated literally) and turned into a case-insensitive regular expression matched against name, description, tags and city. Categories whose name matches are also included, so "cafe" finds every business in the Cafe category.
7. **Geospatial "near me" search.** Locations are GeoJSON points `[longitude, latitude]` with a 2dsphere index. The aggregation stage `$geoNear` returns businesses within the radius (km × 1000 = metres) sorted by distance on a sphere, and the distance is converted to `distanceKm`.
8. **Rating aggregation.** After every review change, `$match` selects the business's reviews and `$group` computes `$avg` and `$sum`; the result is rounded to one decimal and saved on the business.
9. **File upload pipeline.** multer keeps images in memory (only JPEG/PNG/WebP/GIF, max 5 MB each, max 5 per request, max 10 per business). The business is validated first, then files are uploaded to Firestore Storage with a download token; if saving fails, the uploaded files are deleted again.
10. **Pagination and sorting.** `page` and `limit` (max 50) become `skip/limit` in the query; a total count is returned so clients can build page controls.

11. **Centralised error handling.** Controllers throw an `ApiError` with a status code. One error handler also translates Mongoose errors (invalid id → 400, duplicate key → 409, schema validation → 400) and multer errors into JSON.

4.2 Technology stack

Table 6: Technologies used (versions from package-lock.json)

Technology	Version	Used for	In PS?
Node.js	≥ 18 (tested on v26.7.0)	JavaScript runtime for the server	Required
Express.js	5.2.1	HTTP server, routing, middleware	Required
MongoDB (Atlas)	7.0+	Document database	Required
Mongoose	9.10.2	Schemas, validation, queries, aggregation (ODM)	Required
jsonwebtoken	9.0.3	Sign and verify JWTs	Required
firebase-admin	14.5.0	Verify Firebase ID tokens; upload to Firebase Storage	Required
multer	2.4.0	File upload middleware (multipart/form-data)	Required
express-validator	7.3.2	Validation middleware	Required
bcryptjs	3.0.3	Password hashing	Added
helmet, cors, morgan	8.3.0, 2.8.6, 1.12.1	Security headers, cross-origin access, request logs	Added
dotenv	18.0.4	Load settings from .env	Added
Swagger UI (OpenAPI 3)	5.x	Interactive API documentation	Required
Postman	–	Collection of 41 requests	Required
node:test + supertest	built-in, 7.3.0	Automated unit and API tests	Added
Render	–	Hosting the API (Blueprint render.yaml)	Bonus (PS)
Git + GitHub	–	Version control and submission	Required

4.3 Development environment

- Editor: Visual Studio Code. Runtime: Node.js v26.7.0 with npm.
- Database: MongoDB Community Server locally for development; MongoDB Atlas (free M0 cluster) for deployment.
- Firebase project with Authentication and Cloud Storage enabled; service-account credentials supplied through environment variables.
- Tools: Postman for manual testing, Swagger UI in the browser, Git and GitHub for version control, Render for hosting.

5. Problem Statement / Case Study Implementation Details and Snapshots.

5.1 Setting up and running

The source code is in the GitHub repository [\[GitHub repository URL\]](#). The README contains full setup instructions; the short version is:

```
git clone <repository-url> townrate-api
cd townrate-api
npm install
cp .env.example .env          # set MONGO_URI, JWT_SECRET and the Firebase values
npm run seed -- --fresh       # categories + demo users, businesses and reviews
npm run dev                   # http://localhost:5000 (docs at /api-docs)
```

SCREENSHOT 1 — paste here

Capture: the GitHub repository page showing the folder structure (src, tests, postman) and the README.

Screenshot 1: GitHub repository

SCREENSHOT 2 — paste here

Capture: the terminal after `npm run seed -- --fresh` and `npm run dev` – the lines "[seed] created 5 users..." and "[server] TownRate API listening on port 5000".

Screenshot 2: Seeding the database and starting the server

5.2 Key parts of the code

Routes: URL + middleware chain + controller

Each route lists its middleware in order. Static paths such as `/search` are declared before `/:id`, otherwise Express would treat the word "search" as an id. multer runs before validation so that text fields of a multipart form are available to the validators.

```
src/routes/businesses.js
const router = require('express').Router();
const ctrl = require('../controllers/businessController');
const validate = require('../middleware/validate');
const { protect, optionalAuth, authorize } = require('../middleware/auth');
const { uploadPhotos } = require('../middleware/upload');
const v = require('../validators');

const ownerOnly = [protect, authorize('owner', 'admin')];
```

```
// Static paths must come before /:id
router.get('/search', optionalAuth, validate(v.business.search), ctrl.searchBusinesses);
router.get('/near', validate(v.business.near), ctrl.nearBusinesses);

router.get('/', optionalAuth, validate(v.business.list), ctrl.listBusinesses);
// multer runs before validation so multipart text fields are available on req.body
router.post('/', ...ownerOnly, uploadPhotos, validate(v.business.create), ctrl.createBusiness);

router.get('/:id', validate(v.business.id), ctrl.getBusiness);
router.put('/:id', ...ownerOnly, uploadPhotos, validate(v.business.update), ctrl.updateBusiness);
router.delete('/:id', ...ownerOnly, validate(v.business.id), ctrl.deleteBusiness);

router.post('/:id/photos', ...ownerOnly, uploadPhotos, validate(v.business.id), ctrl.addPhotos);
router.delete('/:id/photos/:photoId', ...ownerOnly, validate(v.business.photo), ctrl.deletePhoto);

module.exports = router;
```

JWT protection middleware

src/middleware/auth.js (protect)

```
// Requires a valid app JWT. Loads the user so role changes/deletions take effect immediately.
async function protect(req, _res, next) {
  const token = extractToken(req);
  if (!token) throw ApiError.unauthorized('No token provided');

  let payload;
  try {
    payload = jwt.verify(token, env.jwtSecret);
  } catch (err) {
    throw ApiError.unauthorized(err.name === 'TokenExpiredError' ? 'Token expired' : 'Invalid token');
  }

  const user = await User.findById(payload.sub);
  if (!user) throw ApiError.unauthorized('User no longer exists');
  req.user = user;
  next();
}
```

The token is verified with the secret; the user is then loaded from the database so that a deleted user or a changed role takes effect immediately.

Keyword search

src/controllers/businessController.js (searchBusinesses)

```
// GET /api/businesses/search?keyword=cafe&category=&city=&minRating=
// Case-insensitive partial match on name, description, tags, city and category name.
async function searchBusinesses(req, res) {
  const filter = await buildFilters(req.query, req.user);
  const keyword = (req.query.keyword || '').trim();

  if (keyword) {
    const rx = new RegExp(escapeRegex(keyword), 'i');
    const matchingCategories = await Category.find({ name: rx }).select('_id').lean();
    filter.$or = [
      { name: rx },
      { description: rx },
      { tags: rx },
      { 'address.city': rx },
      { category: { $in: matchingCategories.map((c) => c._id) } },
    ];
  }

  await paginatedFind(res, filter, { sort: 'rating', ...req.query });
}
```

"Near me" geo search

src/controllers/businessController.js (nearBusinesses, query part)

```
async function nearBusinesses(req, res) {
  const lat = Number(req.query.lat);
```

```

const lng = Number(req.query.lng);
const radiusKm = Number(req.query.radius) || 5;
const limit = Math.min(50, parseInt(req.query.limit, 10) || 20);

const query = {};
if (req.query.category) {
  query.category = (await findCategoryId(req.query.category)) || new mongoose.Types.ObjectId();
}
if (req.query.minRating) query.averageRating = { $gte: Number(req.query.minRating) };

const results = await Business.aggregate([
  {
    $geoNear: {
      near: { type: 'Point', coordinates: [lng, lat] },
      distanceField: 'distanceMeters',
      maxDistance: radiusKm * 1000,
      spherical: true,
      query,
    },
  },
  { $limit: limit },
  { $addFields: { distanceKm: { $round: [{ $divide: ['$distanceMeters', 1000] }, 2] } } },
  { $project: { distanceMeters: 0, __v: 0 } },
]);
await Business.populate(results, POPULATE);

```

Rating aggregation

src/models/Review.js

```

// One review per user per business
reviewSchema.index({ business: 1, user: 1 }, { unique: true });

// Recompute the cached rating on the business from its reviews.
reviewSchema.statics.syncBusinessRating = async function syncBusinessRating(businessId) {
  const [agg] = await this.aggregate([
    { $match: { business: new mongoose.Types.ObjectId(String(businessId)) } },
    { $group: { _id: null, avg: { $avg: '$rating' }, count: { $sum: 1 } } },
  ]);
  const stats = roundRatingStats(agg);
  await mongoose.model('Business').findByIdAndUpdate(businessId, stats);
  return stats;
};

```

Photo upload to Firebase Storage

src/services/storage.js (uploadToFirebase)

```

async function uploadToFirebase(file, key) {
  const bucket = getFirebaseApp().storage().bucket();
  const token = crypto.randomUUID();
  await bucket.file(key).save(file.buffer, {
    resumable: false,
    metadata: {
      contentType: file.mimetype,
      // Download token gives a stable public URL without making the bucket public
      metadata: { firebaseStorageDownloadTokens: token },
    },
  });
  const url = `https://firebasestorage.googleapis.com/v0/b/${bucket.name}/o/${
    encodeURIComponent(key)}?alt=media&token=${token}`;
  return { url, path: key, provider: 'firebase' };
}

```

```
src/controllers/businessController.js (saveWithPhotos)
// Upload request files and attach them. If saving fails, roll back the uploads.
async function saveWithPhotos(business, files = []) {
  if (business.photos.length + files.length > env.maxPhotosPerBusiness) {
    throw ApiError.badRequest(`A business can have at most ${env.maxPhotosPerBusiness} photos`);
  }
  await business.validate();
  const uploaded = await storage.uploadFiles(files, `businesses/${business._id}`);
  business.photos.push(...uploaded);
  try {
    await business.save();
  } catch (err) {
    await Promise.all(uploaded.map(storage.deleteFile));
    throw err;
  }
  return business.populate(POPULATE);
}
```

The business is validated before any file is uploaded, and uploaded files are deleted again if the database save fails, so storage never fills up with orphaned photos.

5.3 Snapshots of the API in use

The screenshots follow the example user flow from the PS and then show the remaining features. Requests are sent with the Postman collection (seeded accounts use the password `password123`).

SCREENSHOT 3 — paste here

Capture: the browser at `/api/health` showing `"database": "connected"`, `"firebaseAuth": "enabled"` and `"photoStorage": "firebase"`.

Screenshot 3: Health check (local)

SCREENSHOT 4 — paste here

Capture: Postman – `POST /api/auth/login` as `alice@townrate.dev` with the 200 response containing `token` and `user`.

Screenshot 4: Login returns a JWT

SCREENSHOT 5 — paste here

Capture: Postman – PS flow step 1: `GET /api/businesses/search?keyword=cafe` returning the cafes sorted by rating.

Screenshot 5: PS flow step 1 – search cafes

SCREENSHOT 6 — paste here

Capture: Postman – PS flow step 2: `POST /api/reviews` (Bearer token) returning 201 with the review and the new `businessRating`.

Screenshot 6: PS flow step 2 – leave a review

SCREENSHOT 7 — paste here

Capture: Postman – PS flow step 3: `GET /api/reviews/business/:id` showing the reviews and the `summary.breakdown` of 1–5 stars.

Screenshot 7: PS flow step 3 – view reviews for a business

SCREENSHOT 8 — paste here

Capture: Postman – `POST /api/businesses` as owner (form-data with a `photos` file) returning 201, with `photos[0].provider = "firebase"`.

Screenshot 8: Create a business with a photo

SCREENSHOT 9 — paste here

Capture: Firebase console → Storage, folder `businesses/<businessId>/` containing the uploaded photo.

Screenshot 9: *Photo stored in Firebase Storage*

SCREENSHOT 10 — paste here

Capture: Postman – `GET /api/businesses/near?lat=19.0596&lng=72.8295&radius=5` with results sorted by `distanceKm`.

Screenshot 10: *"Near me" search*

SCREENSHOT 11 — paste here

Capture: Postman – owner `POST /api/reviews/:id/response` returning the review with `ownerResponse.text`.

Screenshot 11: *Owner response to a review*

SCREENSHOT 12 — paste here

Capture: Postman – an error case, e.g. a second review on the same business returning 409, or a user creating a business returning 403.

Screenshot 12: *Error handling example*

SCREENSHOT 13 — paste here

Capture: MongoDB Atlas → Browse Collections → `businesses`, one document showing `location`, `photos` and `averageRating`.

Screenshot 13: Business document in MongoDB Atlas

5.4 API documentation

Swagger UI is served by the API itself at `/api-docs` (raw OpenAPI JSON at `/api-docs.json`). It lists every endpoint with parameters, request bodies and response codes, and the **Authorize** button lets you paste a JWT and try protected endpoints from the browser.

The **Postman collection** (`postman/TownRate.postman_collection.json`) has 41 requests in folders: 0. Health (1), 1. Auth (8), 2. Categories (5), 3. Businesses (15), 4. Reviews (11), 5. Cleanup (1). Login requests save tokens into collection variables and create requests save ids, so the whole collection can run top to bottom in the Collection Runner. Each request has a test that checks the status code.

SCREENSHOT 14 — paste here

Capture: Swagger UI at `/api-docs` showing the endpoint groups (Auth, Businesses, Photos, Reviews, Categories).

Screenshot 14: Swagger UI

SCREENSHOT 15 — paste here

Capture: Postman Collection Runner summary after running the whole collection (41 requests).

Screenshot 15: Postman Collection Runner result

5.5 Deployment on Render

The repository contains a Render Blueprint (`render.yaml`): build command `npm ci`, start command `npm start`, health check `/api/health`. The database is a MongoDB Atlas free cluster, and the secrets (MongoDB URI, JWT secret, Firebase service-account values) are set as environment variables in the Render dashboard, never committed to Git.

Live URL: **[Render URL, e.g. <https://townrate-api.onrender.com>]**

SCREENSHOT 16 — paste here

Capture: Render dashboard – the townrate-api service with status "Live" and the deploy log ending in "TownRate API listening".

Screenshot 16: Render service live

SCREENSHOT 17 — paste here

Capture: the deployed `<Render URL>/api/health` response in the browser.

Screenshot 17: Health check on Render

6. Problem Statement / Case Study Results and Conclusion.

6.1 Results

Table 7: Features and the evidence for each

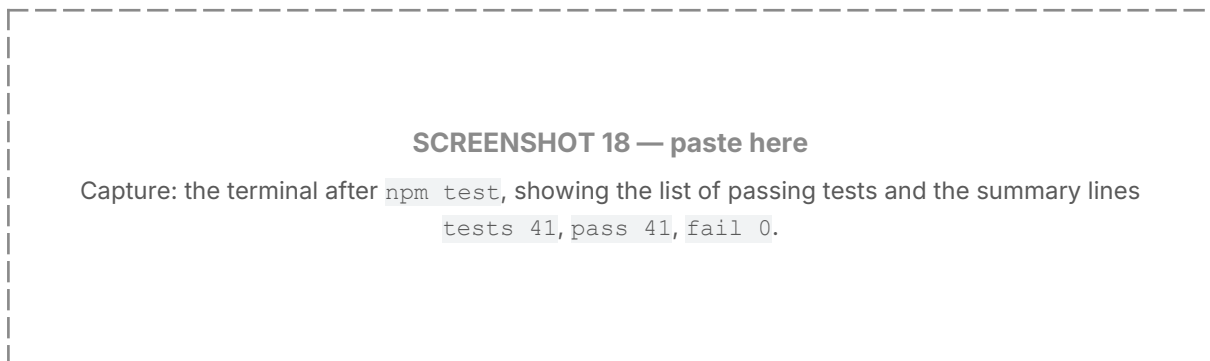
Feature	Result	Evidence
Register / login with JWT	Token returned; protected routes reject missing or invalid tokens with 401	Screenshot 4
Business profiles with photos	Owner creates a business; photos stored in Firebase Storage	Screenshots 8, 9
Search by keyword, category, city	"cafe" returns businesses in the Cafe category and those mentioning cafe	Screenshot 5
"Near me" search	Results sorted by distance with distanceKm	Screenshot 10

Feature	Result	Evidence
Ratings and reviews	Average rating and count update after each review; star breakdown returned	Screenshots 6, 7
Owner response to reviews	Owner of the business can reply; others get 403	Screenshot 11
Validation and error handling	Bad input → 422 with field errors; duplicates → 409; wrong role → 403	Screenshot 12
Documentation	Swagger UI and Postman collection	Screenshots 14, 15
Deployment	Running on Render with MongoDB Atlas	Screenshots 16, 17

6.2 Testing

The project includes an automated test suite (`npm test`, Node's built-in test runner with `supertest`) of 41 tests: 6 unit tests for the helper functions and an end-to-end suite that runs real HTTP requests against a test MongoDB database. The end-to-end tests cover registration and login, role and ownership checks, validation errors, multipart photo upload and deletion, keyword and geo search, rating recalculation, owner responses, categories and error handling.

Expected result: all tests pass (`fail 0`). Screenshot 18 shows the actual output.



Screenshot 18: Automated test results

6.3 Findings

- Splitting the code into routes, middleware, controllers and models kept each file small and made it easy to add the optional features without touching unrelated code.
- Letting MongoDB do the heavy work (2dsphere index for distance, aggregation for averages, unique compound index for "one review per user") is simpler and safer than doing the same in JavaScript.
- Validating input in middleware means controllers can assume clean data, and users get a clear message for every wrong field.

- Caching the average rating on the business makes listing and sorting fast, at the cost of updating it on every review change.

6.4 Limitations and future scope

- Keyword search uses regular expressions; for very large data a MongoDB Atlas Search or text index with relevance ranking would scale better.
- Tokens cannot be revoked before they expire; refresh tokens and a logout blacklist could be added.
- There is no rate limiting yet; `express-rate-limit` on login and review endpoints would reduce abuse.
- Photos are not resized; an image-processing step (for example sharp) could create thumbnails.
- A frontend (web or mobile) using the Firebase client SDK would complete the product.

6.5 Conclusion

This case study delivered a complete backend for TownRate that meets every requirement of the problem statement, including both optional features. Business owners can publish profiles with photos stored in Firebase Storage, users can find businesses by keyword, category, city or distance, and the rating system stays consistent through MongoDB aggregation and indexes. JWT and Firebase authentication, role checks, validation and centralised error handling make the API safe to use, and Swagger, Postman, automated tests and the Render deployment make it easy to understand, verify and run.

7. References

- Node.js documentation – <https://nodejs.org/docs/latest/api/>
- Express 5 documentation (routing, middleware, error handling) – <https://expressjs.com/>
- MongoDB Manual: data modelling, indexes, geospatial queries (\$geoNear, 2dsphere), aggregation – <https://www.mongodb.com/docs/manual/>
- Mongoose documentation (schemas, middleware, populate) – <https://mongoosejs.com/docs/>
- RFC 7519 – JSON Web Token (JWT) – <https://datatracker.ietf.org/doc/html/rfc7519>
- jsonwebtoken package – <https://github.com/auth0/node-jwebtoken>
- Firebase Admin SDK: verify ID tokens; Cloud Storage – <https://firebase.google.com/docs/admin/setup>
- multer documentation – <https://github.com/expressjs/multer>
- express-validator documentation – <https://express-validator.github.io/docs/>

- OpenAPI Specification 3.0 and Swagger UI – <https://swagger.io/specification/>
- Postman Learning Center (collections, variables, Collection Runner) – <https://learning.postman.com/>
- Render documentation (Blueprints, web services, environment variables) – <https://render.com/docs>
- OWASP Password Storage Cheat Sheet – <https://cheatsheetseries.owasp.org/>
- Project repository and README – **[GitHub repository URL]**

Appendix A – Complete Source Code

The listings below are the complete source of the project as submitted (package-lock.json, the Postman collection JSON and node_modules are omitted for length). Files used only for an optional serverless host (api/index.js, vercel.json) are not needed for the Render deployment and are not reproduced.

package.json

```
{
  "name": "townrate-api",
  "version": "1.0.0",
  "description": "TownRate - Local Business Review Platform backend (Node.js, Express, MongoDB, Firebase)",
  "main": "src/server.js",
  "scripts": {
    "start": "node src/server.js",
    "dev": "node --watch src/server.js",
    "seed": "node scripts/seed.js",
    "test": "node --test --test-concurrency=1 \"tests/**/*.test.js\""
  },
  "engines": {
    "node": ">=18"
  },
  "keywords": [
    "express",
    "mongodb",
    "mongoose",
    "firebase",
    "jwt",
    "reviews"
  ],
  "license": "MIT",
  "dependencies": {
    "bcryptjs": "^3.0.3",
    "cors": "^2.8.6",
    "dotenv": "^18.0.4",
    "express": "^5.2.1",
    "express-validator": "^7.3.2",
    "firebase-admin": "^14.5.0",
    "helmet": "^8.3.0",
    "jsonwebtoken": "^9.0.3",
    "mongoose": "^9.10.2",
    "morgan": "^1.12.1",
    "multer": "^2.4.0"
  },
  "devDependencies": {
    "supertest": "^7.3.0"
  }
}
```

.env.example

```
# Server
PORT=5000
NODE_ENV=development
# Public base URL, used to build links for locally stored photos
BASE_URL=http://localhost:5000

# MongoDB
MONGO_URI=mongodb://127.0.0.1:27017/townrate

# JWT
JWT_SECRET=change-me-to-a-long-random-string
JWT_EXPIRES_IN=7d

# Firebase (optional - without these, photos are stored on local disk
# and POST /api/auth/firebase returns 503)
# Option A: paste service account fields
FIREBASE_PROJECT_ID=
FIREBASE_CLIENT_EMAIL=
# Keep the quotes; \n sequences are converted to newlines automatically
FIREBASE_PRIVATE_KEY=""
# Option B: path to a service account JSON file
# GOOGLE_APPLICATION_CREDENTIALS=./serviceAccountKey.json
FIREBASE_STORAGE_BUCKET=your-project.appspot.com

# Uploads
MAX_FILE_SIZE_MB=5
MAX_PHOTOS_PER_BUSINESS=10
```

render.yaml

```
# Render Blueprint - https://render.com/docs/blueprint-spec
# Set MONGO_URI (e.g. MongoDB Atlas) and Firebase variables in the dashboard.
services:
  - type: web
    name: townrate-api
    runtime: node
    plan: free
    buildCommand: npm ci
    startCommand: npm start
    healthCheckPath: /api/health
    envVars:
      - key: NODE_ENV
        value: production
      - key: JWT_SECRET
        generateValue: true
      - key: JWT_EXPIRES_IN
        value: 7d
      - key: MONGO_URI
        sync: false
      - key: BASE_URL
        sync: false
      - key: FIREBASE_PROJECT_ID
        sync: false
      - key: FIREBASE_CLIENT_EMAIL
        sync: false
      - key: FIREBASE_PRIVATE_KEY
        sync: false
      - key: FIREBASE_STORAGE_BUCKET
        sync: false
```

src/server.js

```
const app = require('./app');
const env = require('./config/env');
const { connectDB, disconnectDB } = require('./config/db');

async function start() {
  try {
    await connectDB();
  } catch (err) {
    console.error(`[db] connection failed: ${err.message}`);
    process.exit(1);
  }

  const server = app.listen(env.port, () => {
    console.log(`[server] TownRate API listening on port ${env.port} (${env.nodeEnv})`);
    console.log(`[server] docs at ${env.baseUrl}/api-docs`);
  });

  const shutdown = (signal) => {
    console.log(`[server] ${signal} received, shutting down`);
    server.close(async () => {
      await disconnectDB();
      process.exit(0);
    });
  };
  process.on('SIGINT', () => shutdown('SIGINT'));
  process.on('SIGTERM', () => shutdown('SIGTERM'));
}

start();
```

src/app.js

```
const express = require('express');
const cors = require('cors');
const helmet = require('helmet');
const morgan = require('morgan');
const mongoose = require('mongoose');
const env = require('./config/env');
const { isFirebaseEnabled } = require('./config/firebase');
const { LOCAL_DIR, storageMode } = require('./services/storage');
const openapi = require('./docs/openapi');
const swaggerPage = require('./docs/swaggerPage');
const routes = require('./routes');
const { notFound, errorHandler } = require('./middleware/errorHandler');

const app = express();

app.use(
  helmet({
    // Allow locally stored photos to be embedded by a frontend on another origin
    crossOriginResourcePolicy: { policy: 'cross-origin' },
```

```

// Swagger UI assets come from jsDelivr (works on Vercel, where node_modules assets aren't served)
contentSecurityPolicy: {
  directives: {
    scriptSrc: ['"self"', 'https://cdn.jsdelivr.net'],
    styleSrc: ['"self"', "'unsafe-inline'", 'https://cdn.jsdelivr.net'],
    imgSrc: ['"self"', 'data:', 'https:'],
  },
},
))
);
app.use(cors());
app.use(express.json({ limit: '1mb' }));
app.use(express.urlencoded({ extended: true }));
if (env.nodeEnv !== 'test') app.use(morgan(env.nodeEnv === 'production' ? 'combined' : 'dev'));

app.use('/uploads', express.static(LOCAL_DIR));

app.get('/', (_req, res) => {
  res.json({ success: true, name: 'TownRate API', docs: '/api-docs', health: '/api/health' });
});

app.get('/api/health', (_req, res) => {
  const dbUp = mongoose.connection.readyState === 1;
  res.status(dbUp ? 200 : 503).json({
    success: dbUp,
    status: dbUp ? 'ok' : 'degraded',
    database: dbUp ? 'connected' : 'disconnected',
    firebaseAuth: isFirebaseEnabled() ? 'enabled' : 'disabled',
    photoStorage: storageMode(),
    uptime: Math.round(process.uptime()),
  });
});

app.get('/api-docs.json', (_req, res) => res.json(openapi));
app.use('/api-docs', swaggerPage);

app.use('/api', routes);

app.use(notFound);
app.use(errorHandler);

module.exports = app;

```

src/config/env.js

```

require('dotenv').config({ quiet: true });

const env = {
  port: Number(process.env.PORT) || 5000,
  nodeEnv: process.env.NODE_ENV || 'development',
  baseUrl: process.env.BASE_URL || `http://localhost:${Number(process.env.PORT) || 5000}`,
  mongoUri: process.env.MONGO_URI || 'mongodb://127.0.0.1:27017/townrate',
  jwtSecret: process.env.JWT_SECRET,
  jwtExpiresIn: process.env.JWT_EXPIRES_IN || '7d',
  firebase: {
    projectId: process.env.FIREBASE_PROJECT_ID,
    clientEmail: process.env.FIREBASE_CLIENT_EMAIL,
    privateKey: process.env.FIREBASE_PRIVATE_KEY
      ? process.env.FIREBASE_PRIVATE_KEY.replace(/\n/g, '\n')
      : undefined,
    credentialsFile: process.env.GOOGLE_APPLICATION_CREDENTIALS,
    storageBucket: process.env.FIREBASE_STORAGE_BUCKET,
  },
  maxFileSizeMb: Number(process.env.MAX_FILE_SIZE_MB) || 5,
  maxPhotosPerBusiness: Number(process.env.MAX_PHOTOS_PER_BUSINESS) || 10,
  // Vercel functions have a read-only filesystem, so local photo storage is unavailable
  isServerless: Boolean(process.env.VERCEL),
};

if (!env.jwtSecret) {
  if (env.nodeEnv === 'production') {
    throw new Error('JWT_SECRET must be set in production');
  }
  env.jwtSecret = 'dev-only-insecure-secret';
  if (env.nodeEnv !== 'test') {
    console.warn('[config] JWT_SECRET not set - using an insecure development secret');
  }
}

module.exports = env;

```

src/config/db.js

```
const mongoose = require('mongoose');
const env = require('./env');

// Shared across calls so serverless invocations (Vercel) reuse one connection
// instead of opening a new one per request.
let connecting = null;

async function connectDB(uri = env.mongoUri) {
  if (mongoose.connection.readyState === 1) return mongoose.connection;
  if (!connecting) {
    mongoose.set('strictQuery', true);
    connecting = mongoose
      .connect(uri, { serverSelectionTimeoutMS: 5000, maxPoolSize: 10 })
      .then(() => {
        if (env.nodeEnv !== 'test') {
          console.log(`[db] connected to ${mongoose.connection.host}/${mongoose.connection.name}`);
        }
        return mongoose.connection;
      })
      .catch((err) => {
        connecting = null;
        throw err;
      });
  }
  return connecting;
}

async function disconnectDB() {
  connecting = null;
  await mongoose.disconnect();
}

module.exports = { connectDB, disconnectDB };
```

src/config/firebase.js

```
const env = require('./env');

// Firebase is optional. When credentials are missing the app still runs:
// photos go to local disk and the Firebase login exchange is disabled.
let app = null;
let initialised = false;

function getFirebaseApp() {
  if (initialised) return app;
  initialised = true;

  const { projectId, clientEmail, privateKey, credentialsFile, storageBucket } = env.firebase;
  const hasInlineCreds = projectId && clientEmail && privateKey;
  if (!hasInlineCreds && !credentialsFile) return null;

  try {
    const admin = require('firebase-admin');
    const credential = hasInlineCreds
      ? admin.credential.cert({ projectId, clientEmail, privateKey })
      : admin.credential.applicationDefault();
    app = admin.initializeApp({ credential, storageBucket });
    console.log('[firebase] initialised');
  } catch (err) {
    console.error('[firebase] failed to initialise, continuing without it:', err.message);
    app = null;
  }
  return app;
}

function isFirebaseEnabled() {
  return getFirebaseApp() !== null;
}

function isStorageEnabled() {
  return isFirebaseEnabled() && Boolean(env.firebase.storageBucket);
}

module.exports = { getFirebaseApp, isFirebaseEnabled, isStorageEnabled };
```

src/models/User.js

```
const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');
```

```

const ROLES = ['user', 'owner', 'admin'];

const userSchema = new mongoose.Schema(
  {
    name: { type: String, required: true, trim: true, maxlength: 80 },
    email: {
      type: String,
      required: true,
      unique: true,
      lowercase: true,
      trim: true,
      match: [/^\S+@\S+\.\S+$/, 'Invalid email address'],
    },
    // Not required: users who sign in only through Firebase have no local password.
    password: { type: String, minlength: 6, select: false },
    role: { type: String, enum: ROLES, default: 'user' },
    firebaseUid: { type: String, unique: true, sparse: true },
    avatarUrl: String,
  },
  { timestamps: true }
);

userSchema.pre('save', async function hashPassword() {
  if (!this.isModified('password') || !this.password) return;
  this.password = await bcrypt.hash(this.password, 10);
});

userSchema.methods.comparePassword = function comparePassword(candidate) {
  if (!this.password) return Promise.resolve(false);
  return bcrypt.compare(candidate, this.password);
};

userSchema.set('toJSON', {
  transform(_doc, ret) {
    delete ret.password;
    delete ret.__v;
    return ret;
  },
});

module.exports = mongoose.model('User', userSchema);
module.exports.ROLES = ROLES;

```

src/models/Business.js

```

const mongoose = require('mongoose');

const photoSchema = new mongoose.Schema(
  {
    url: { type: String, required: true },
    // Storage path/key, needed to delete the file later
    path: { type: String, required: true },
    provider: { type: String, enum: ['firebase', 'local'], required: true },
  },
  { timestamps: { createdAt: true, updatedAt: false } }
);

const businessSchema = new mongoose.Schema(
  {
    owner: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true, index: true },
    name: { type: String, required: true, trim: true, maxlength: 120 },
    description: { type: String, trim: true, maxlength: 2000 },
    category: { type: mongoose.Schema.Types.ObjectId, ref: 'Category', required: true, index: true },
    tags: [{ type: String, trim: true, lowercase: true }],
    address: {
      street: { type: String, trim: true },
      city: { type: String, trim: true, required: true },
      state: { type: String, trim: true },
      zip: { type: String, trim: true },
      country: { type: String, trim: true, default: 'India' },
    },
    // GeoJSON point, coordinates are [longitude, latitude]
    location: {
      type: { type: String, enum: ['Point'], default: 'Point' },
      coordinates: {
        type: [Number],
        validate: {
          validator: (v) => !v || v.length === 0 || (v.length === 2 && v[0] >= -180 && v[0] <= 180 && v[1]
          >= -90 && v[1] <= 90),
          message: 'coordinates must be [longitude, latitude]',
        },
      },
    },
  },

```

```

    },
    phone: { type: String, trim: true },
    email: { type: String, trim: true, lowercase: true },
    website: { type: String, trim: true },
    photos: [photoSchema],
    averageRating: { type: Number, default: 0, min: 0, max: 5 },
    reviewCount: { type: Number, default: 0, min: 0 },
  },
  { timestamps: true }
);

businessSchema.index({ location: '2dsphere' });
businessSchema.index({ 'address.city': 1 });
businessSchema.index({ averageRating: -1 });

// A Point with no coordinates breaks the 2dsphere index, so drop it entirely.
businessSchema.pre('validate', function cleanLocation() {
  if (this.location && (!this.location.coordinates || this.location.coordinates.length === 0)) {
    this.location = undefined;
  }
});

businessSchema.set('toJSON', { versionKey: false });

module.exports = mongoose.model('Business', businessSchema);

```

src/models/Review.js

```

const mongoose = require('mongoose');
const { roundRatingStats } = require('../utils/query');

const reviewSchema = new mongoose.Schema(
  {
    business: { type: mongoose.Schema.Types.ObjectId, ref: 'Business', required: true, index: true },
    user: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
    rating: { type: Number, required: true, min: 1, max: 5 },
    title: { type: String, trim: true, maxlength: 120 },
    comment: { type: String, trim: true, maxlength: 2000 },
    ownerResponse: {
      text: { type: String, trim: true, maxlength: 1000 },
      respondedAt: Date,
    },
  },
  { timestamps: true }
);

// One review per user per business
reviewSchema.index({ business: 1, user: 1 }, { unique: true });

// Recompute the cached rating on the business from its reviews.
reviewSchema.statics.syncBusinessRating = async function syncBusinessRating(businessId) {
  const [agg] = await this.aggregate([
    { $match: { business: new mongoose.Types.ObjectId(String(businessId)) } },
    { $group: { _id: null, avg: { $avg: '$rating' }, count: { $sum: 1 } } },
  ]);
  const stats = roundRatingStats(agg);
  await mongoose.model('Business').findByIdAndUpdate(businessId, stats);
  return stats;
};

reviewSchema.set('toJSON', { versionKey: false });

module.exports = mongoose.model('Review', reviewSchema);

```

src/models/Category.js

```
const mongoose = require('mongoose');
const { slugify } = require('../utils/query');

const categorySchema = new mongoose.Schema(
  {
    name: { type: String, required: true, unique: true, trim: true, maxlength: 50 },
    slug: { type: String, unique: true, lowercase: true },
    description: { type: String, maxlength: 200 },
    icon: String,
  },
  { timestamps: true }
);

categorySchema.pre('validate', function setSlug() {
  if (this.isModified('name') || !this.slug) this.slug = slugify(this.name);
});

categorySchema.set('toJSON', { versionKey: false });

module.exports = mongoose.model('Category', categorySchema);
```

src/middleware/auth.js

```
const jwt = require('jsonwebtoken');
const env = require('../config/env');
const User = require('../models/User');
const ApiError = require('../utils/ApiError');

function signToken(user) {
  return jwt.sign({ sub: user._id.toString(), role: user.role }, env.jwtSecret, {
    expiresIn: env.jwtExpiresIn,
  });
}

function extractToken(req) {
  const header = req.headers.authorization || '';
  const [scheme, token] = header.split(' ');
  return scheme === 'Bearer' && token ? token : null;
}

// Requires a valid app JWT. Loads the user so role changes/deletions take effect immediately.
async function protect(req, _res, next) {
  const token = extractToken(req);
  if (!token) throw ApiError.unauthorized('No token provided');

  let payload;
  try {
    payload = jwt.verify(token, env.jwtSecret);
  } catch (err) {
    throw ApiError.unauthorized(err.name === 'TokenExpiredError' ? 'Token expired' : 'Invalid token');
  }

  const user = await User.findById(payload.sub);
  if (!user) throw ApiError.unauthorized('User no longer exists');
  req.user = user;
  next();
}

// Attaches req.user when a valid token is present, but never rejects.
async function optionalAuth(req, _res, next) {
  const token = extractToken(req);
  if (token) {
    try {
      const payload = jwt.verify(token, env.jwtSecret);
      req.user = await User.findById(payload.sub);
    } catch {
      // ignore - treat as anonymous
    }
  }
  next();
}

function authorize(...roles) {
  return (req, _res, next) => {
    if (!req.user) throw ApiError.unauthorized();
    if (!roles.includes(req.user.role)) {
      throw ApiError.forbidden(`This action requires one of the roles: ${roles.join(', ')}`);
    }
    next();
  };
}
```

```

    };
  }

module.exports = { signToken, protect, optionalAuth, authorize };

```

src/middleware/upload.js

```

const multer = require('multer');
const env = require('../../config/env');
const ApiError = require('../../utils/ApiError');

const ALLOWED_TYPES = ['image/jpeg', 'image/png', 'image/webp', 'image/gif'];
const MAX_FILES_PER_REQUEST = 5;

// Files stay in memory; the storage service decides whether they go to Firebase or disk.
const upload = multer({
  storage: multer.memoryStorage(),
  limits: { fileSize: env.maxFileSizeMb * 1024 * 1024, files: MAX_FILES_PER_REQUEST },
  fileFilter(_req, file, cb) {
    if (ALLOWED_TYPES.includes(file.mimetype)) return cb(null, true);
    cb(ApiError.badRequest(`Unsupported file type ${file.mimetype}. Allowed: ${ALLOWED_TYPES.join(', ')}`));
  },
});

const uploadPhotos = upload.array('photos', MAX_FILES_PER_REQUEST);

module.exports = { uploadPhotos, ALLOWED_TYPES, MAX_FILES_PER_REQUEST };

```

src/middleware/validate.js

```

const { validationResult } = require('express-validator');

// Runs express-validator chains and responds 422 with field errors if any fail.
function validate(chains) {
  return async (req, res, next) => {
    for (const chain of chains) {
      await chain.run(req);
    }
    const result = validationResult(req);
    if (result.isEmpty()) return next();

    res.status(422).json({
      success: false,
      message: 'Validation failed',
      errors: result.array().map((e) => ({ field: e.path, message: e.msg, location: e.location })),
    });
  };
}

module.exports = validate;

```

src/middleware/errorHandler.js

```

const mongoose = require('mongoose');
const multer = require('multer');
const env = require('../../config/env');
const ApiError = require('../../utils/ApiError');

function notFound(req, _res, next) {
  next(ApiError.notFound(`Route not found: ${req.method} ${req.originalUrl}`));
}

// eslint-disable-next-line no-unused-vars
function errorHandler(err, _req, res, _next) {
  let status = err.statusCode || 500;
  let message = err.message || 'Internal server error';
  let details = err.details;

  if (err instanceof mongoose.Error.CastError) {
    status = 400;
    message = `Invalid ${err.path}: ${err.value}`;
  } else if (err instanceof mongoose.Error.ValidationError) {
    status = 400;
    message = 'Validation failed';
    details = Object.values(err.errors).map((e) => ({ field: e.path, message: e.message }));
  } else if (err.code === 11000) {
    status = 409;
    const field = Object.keys(err.keyValue || {})[0] || 'field';
    message = `Duplicate value for ${field}`;
  } else if (err instanceof multer.MulterError) {
    status = 400;
  }
}

```

```

    message = err.code === 'LIMIT_FILE_SIZE'
      ? `File too large (max ${env.maxFileSizeMb}MB)`
      : err.message;
  } else if (err.type === 'entity.parse.failed') {
    status = 400;
    message = 'Malformed JSON body';
  }

  if (status >= 500 && env.nodeEnv !== 'test') console.error(err);

  const body = { success: false, message };
  if (details) body.errors = details;
  if (env.nodeEnv === 'development' && status >= 500) body.stack = err.stack;
  res.status(status).json(body);
}

module.exports = { notFound, errorHandler };

```

src/validators/index.js

```

const { body, param, query } = require('express-validator');

const objectId = (field, location = param) =>
  location(field).isMongoId().withMessage(`${field} must be a valid id`);

// Accept "a,b,c" (multipart forms) or a real array (JSON).
const toArray = (v) => {
  if (v === undefined || v === null || v === '') return [];
  return Array.isArray(v) ? v : String(v).split(',').map((s) => s.trim()).filter(Boolean);
};

const auth = {
  register: [
    body('name').trim().notEmpty().withMessage('name is required').isLength({ max: 80 }),
    body('email').trim().isEmail().withMessage('valid email is required').normalizeEmail(),
    body('password').isLength({ min: 6 }).withMessage('password must be at least 6 characters'),
    body('role').optional().isIn(['user', 'owner']).withMessage('role must be user or owner'),
  ],
  login: [
    body('email').trim().isEmail().withMessage('valid email is required').normalizeEmail(),
    body('password').notEmpty().withMessage('password is required'),
  ],
  firebase: [
    body('idToken').notEmpty().withMessage('idToken is required'),
    body('role').optional().isIn(['user', 'owner']).withMessage('role must be user or owner'),
  ],
};

const coordinates = [
  body('latitude').optional({ values: 'falsy' }).isFloat({ min: -90, max: 90 }).withMessage('latitude must be between -90 and 90').toFloat(),
  body('longitude').optional({ values: 'falsy' }).isFloat({ min: -180, max: 180 }).withMessage('longitude must be between -180 and 180').toFloat(),
  body('longitude').custom((lng, { req }) => {
    const hasLat = req.body.latitude !== undefined && req.body.latitude !== '';
    const hasLng = lng !== undefined && lng !== '';
    if (hasLat !== hasLng) throw new Error('latitude and longitude must be provided together');
    return true;
  }),
];

const businessOptionalFields = [
  body('description').optional().isString().isLength({ max: 2000 }),
  body('tags').optional().customSanitizer(toArray).isArray({ max: 20 }).withMessage('at most 20 tags'),
  body('phone').optional({ values: 'falsy' }).matches(/^\+[0-9]{5,19}$)/.withMessage('invalid phone number'),
  body('email').optional({ values: 'falsy' }).isEmail().withMessage('invalid email'),
  body('website').optional({ values: 'falsy' }).isURL().withMessage('invalid website URL'),
  ...coordinates,
];

const business = {
  create: [
    body('name').trim().notEmpty().withMessage('name is required').isLength({ max: 120 }),
    body('category').trim().notEmpty().withMessage('category (id or slug) is required'),
    body('address.city').trim().notEmpty().withMessage('address.city is required'),
    ...businessOptionalFields,
  ],
  update: [
    objectId('id'),
    body('name').optional().trim().notEmpty().isLength({ max: 120 }),
    body('category').optional().trim().notEmpty(),
  ],
};

```

```

    body('address.city').optional().trim().notEmpty().withMessage('address.city cannot be empty'),
    body('owner').not().exists().withMessage('owner cannot be changed'),
    body(['averageRating', 'reviewCount']).not().exists().withMessage('rating fields are computed and
cannot be set'),
    ...businessOptionalFields,
  ],
  id: [objectId('id')],
  photo: [objectId('id'), objectId('photoId')],
  list: [
    query('minRating').optional().isFloat({ min: 0, max: 5 }).toFloat(),
    query('page').optional().isInt({ min: 1 }),
    query('limit').optional().isInt({ min: 1, max: 50 }),
    query('sort').optional().isIn(['rating', 'reviews', 'newest', 'oldest', 'name']),
    query('owner').optional().custom((v) => v === 'me' || /^[a-f\d]{24}$/i.test(v)).withMessage('owner must
be an id or "me"'),
  ],
  search: [
    query('keyword').optional().trim().isLength({ max: 100 }),
    query('minRating').optional().isFloat({ min: 0, max: 5 }).toFloat(),
    query('page').optional().isInt({ min: 1 }),
    query('limit').optional().isInt({ min: 1, max: 50 }),
    query('keyword').custom((kw, { req }) => {
      if (!kw && !req.query.category && !req.query.city) {
        throw new Error('provide at least one of keyword, category or city');
      }
      return true;
    }),
  ],
  near: [
    query('lat').exists().withMessage('lat is required').isFloat({ min: -90, max: 90 }).toFloat(),
    query('lng').exists().withMessage('lng is required').isFloat({ min: -180, max: 180 }).toFloat(),
    query('radius').optional().isFloat({ min: 0.1, max: 100 }).withMessage('radius must be 0.1-100
km').toFloat(),
    query('limit').optional().isInt({ min: 1, max: 50 }),
  ],
];

const review = {
  create: [
    body('business').isMongoId().withMessage('business must be a valid id'),
    body('rating').isInt({ min: 1, max: 5 }).withMessage('rating must be an integer 1-5').toInt(),
    body('title').optional().trim().isLength({ max: 120 }),
    body('comment').optional().trim().isLength({ max: 2000 }),
  ],
  update: [
    objectId('id'),
    body('rating').optional().isInt({ min: 1, max: 5 }).withMessage('rating must be an integer 1-
5').toInt(),
    body('title').optional().trim().isLength({ max: 120 }),
    body('comment').optional().trim().isLength({ max: 2000 }),
  ],
  id: [objectId('id')],
  forBusiness: [
    objectId('id'),
    query('sort').optional().isIn(['newest', 'oldest', 'highest', 'lowest']),
    query('page').optional().isInt({ min: 1 }),
    query('limit').optional().isInt({ min: 1, max: 50 }),
  ],
  respond: [
    objectId('id'),
    body('text').trim().notEmpty().withMessage('text is required').isLength({ max: 1000 }),
  ],
};

const category = {
  create: [
    body('name').trim().notEmpty().withMessage('name is required').isLength({ max: 50 }),
    body('description').optional().trim().isLength({ max: 200 }),
    body('icon').optional().trim(),
  ],
  update: [
    objectId('id'),
    body('name').optional().trim().notEmpty().isLength({ max: 50 }),
    body('description').optional().trim().isLength({ max: 200 }),
    body('icon').optional().trim(),
  ],
  id: [objectId('id')],
};

module.exports = { auth, business, review, category, toArray };

```

src/controllers/authController.js

```

const User = require('../models/User');
const ApiError = require('../utils/ApiError');
const { signToken } = require('../middleware/auth');

```

```

const { getFirebaseApp, isFirebaseEnabled } = require('../config/firebase');

function authResponse(res, status, user) {
  res.status(status).json({ success: true, data: { token: signToken(user), user } });
}

// POST /api/auth/register
async function register(req, res) {
  const { name, email, password, role = 'user' } = req.body;
  if (await User.exists({ email })) throw ApiError.conflict('Email is already registered');
  const user = await User.create({ name, email, password, role });
  authResponse(res, 201, user);
}

// POST /api/auth/login
async function login(req, res) {
  const { email, password } = req.body;
  const user = await User.findOne({ email }).select('+password');
  if (!user || !(await user.comparePassword(password))) {
    throw ApiError.unauthorized('Invalid email or password');
  }
  authResponse(res, 200, user);
}

// POST /api/auth/firebase
// Exchanges a Firebase ID token (from the client SDK) for an app JWT.
// Links to an existing account by email, or creates one on first sign-in.
async function firebaseLogin(req, res) {
  if (!isFirebaseEnabled()) {
    throw new ApiError(503, 'Firebase authentication is not configured on this server');
  }

  let decoded;
  try {
    decoded = await getFirebaseApp().auth().verifyIdToken(req.body.idToken);
  } catch {
    throw ApiError.unauthorized('Invalid or expired Firebase ID token');
  }
  if (!decoded.email) throw ApiError.badRequest('Firebase account has no email address');

  let user = await User.findOne({ firebaseUid: decoded.uid });
  let status = 200;
  if (!user) {
    user = await User.findOne({ email: decoded.email.toLowerCase() });
    if (user) {
      user.firebaseUid = decoded.uid;
      await user.save();
    } else {
      user = await User.create({
        name: decoded.name || decoded.email.split('@')[0],
        email: decoded.email,
        firebaseUid: decoded.uid,
        avatarUrl: decoded.picture,
        role: req.body.role || 'user',
      });
      status = 201;
    }
  }
  authResponse(res, status, user);
}

// GET /api/auth/me
function me(req, res) {
  res.json({ success: true, data: req.user });
}

module.exports = { register, login, firebaseLogin, me };

```

src/controllers/businessController.js

```

const mongoose = require('mongoose');
const Business = require('../models/Business');
const Category = require('../models/Category');
const Review = require('../models/Review');
const ApiError = require('../utils/ApiError');
const env = require('../config/env');
const storage = require('../services/storage');
const {
  escapeRegex,
  getPagination,
  buildPaginationMeta,
  getSort,

```

```

} = require('../utils/query');

const POPULATE = [
  { path: 'category', select: 'name slug icon' },
  { path: 'owner', select: 'name' },
];

const UPDATABLE = ['name', 'description', 'tags', 'phone', 'email', 'website'];
const ADDRESS_FIELDS = ['street', 'city', 'state', 'zip', 'country'];

// Accepts a category id, slug or name.
async function resolveCategory(value) {
  const v = String(value).trim();
  const filter = mongoose.isValidObjectId(v)
    ? { _id: v }
    : { $or: [{ slug: v.toLowerCase() }, { name: new RegExp(`^${escapeRegex(v)}$`, 'i') }] };
  const category = await Category.findOne(filter);
  if (!category) throw ApiError.badRequest(`Unknown category: ${v}`);
  return category;
}

// Same as above but for query filters: returns the id, or null if the category doesn't exist.
async function findCategoryId(value) {
  try {
    return (await resolveCategory(value))._id;
  } catch {
    return null;
  }
}

async function loadOwnedBusiness(req) {
  const business = await Business.findById(req.params.id);
  if (!business) throw ApiError.notFound('Business not found');
  const isOwner = business.owner.equals(req.user._id);
  if (!isOwner && req.user.role !== 'admin') {
    throw ApiError.forbidden('Only the business owner can modify this business');
  }
  return business;
}

function applyFields(business, body) {
  for (const key of UPDATABLE) {
    if (body[key] !== undefined) business[key] = body[key];
  }
  if (body.address && typeof body.address === 'object') {
    for (const key of ADDRESS_FIELDS) {
      if (body.address[key] !== undefined) business.set(`address.${key}`, body.address[key]);
    }
  }
  if (body.latitude !== undefined && body.latitude !== '' && body.longitude !== undefined && body.longitude !== '') {
    business.location = { type: 'Point', coordinates: [Number(body.longitude), Number(body.latitude)] };
  }
}

// Upload request files and attach them. If saving fails, roll back the uploads.
async function saveWithPhotos(business, files = []) {
  if (business.photos.length + files.length > env.maxPhotosPerBusiness) {
    throw ApiError.badRequest(`A business can have at most ${env.maxPhotosPerBusiness} photos`);
  }
  await business.validate();
  const uploaded = await storage.uploadFiles(files, `businesses/${business._id}`);
  business.photos.push(...uploaded);
  try {
    await business.save();
  } catch (err) {
    await Promise.all(uploaded.map(storage.deleteFile));
    throw err;
  }
  return business.populate(POPULATE);
}

async function buildFilters(q, user) {
  const filter = {};
  if (q.category) {
    const id = await findCategoryId(q.category);
    // Unknown category: force an empty result rather than ignoring the filter
    filter.category = id || new mongoose.Types.ObjectId();
  }
  if (q.city) filter['address.city'] = new RegExp(`^${escapeRegex(q.city.trim())}$`, 'i');
  if (q.minRating !== undefined) filter.averageRating = { $gte: Number(q.minRating) };
  if (q.owner) {

```

```

    if (q.owner === 'me') {
      if (!user) throw ApiError.unauthorized('Log in to list your own businesses');
      filter.owner = user._id;
    } else {
      filter.owner = q.owner;
    }
  }
  return filter;
}

async function paginatedFind(res, filter, query) {
  const { page, limit, skip } = getPagination(query);
  const [items, total] = await Promise.all([
    Business.find(filter).sort(getSort(query.sort)).skip(skip).limit(limit).populate(POPULATE),
    Business.countDocuments(filter),
  ]);
  res.json({ success: true, data: items, pagination: buildPaginationMeta(total, page, limit) });
}

// GET /api/businesses?category=&city=&minRating=&owner=&sort=&page=&limit=
async function listBusinesses(req, res) {
  const filter = await buildFilters(req.query, req.user);
  await paginatedFind(res, filter, req.query);
}

// GET /api/businesses/search?keyword=cafe&category=&city=&minRating=
// Case-insensitive partial match on name, description, tags, city and category name.
async function searchBusinesses(req, res) {
  const filter = await buildFilters(req.query, req.user);
  const keyword = (req.query.keyword || '').trim();

  if (keyword) {
    const rx = new RegExp(escapeRegex(keyword), 'i');
    const matchingCategories = await Category.find({ name: rx }).select('_id').lean();
    filter.$or = [
      { name: rx },
      { description: rx },
      { tags: rx },
      { 'address.city': rx },
      { category: { $in: matchingCategories.map((c) => c._id) } },
    ];
  }

  await paginatedFind(res, filter, { sort: 'rating', ...req.query });
}

// GET /api/businesses/near?lat=&lng=&radius=5&category=&limit=
// Sorted by distance; each result includes distanceKm.
// Express 5 exposes req.query as a read-only getter, so validator sanitizers
// can't coerce it in place - convert here instead.
async function nearBusinesses(req, res) {
  const lat = Number(req.query.lat);
  const lng = Number(req.query.lng);
  const radiusKm = Number(req.query.radius) || 5;
  const limit = Math.min(50, parseInt(req.query.limit, 10) || 20);

  const query = {};
  if (req.query.category) {
    query.category = (await findCategoryId(req.query.category)) || new mongoose.Types.ObjectId();
  }
  if (req.query.minRating) query.averageRating = { $gte: Number(req.query.minRating) };

  const results = await Business.aggregate([
    {
      $geoNear: {
        near: { type: 'Point', coordinates: [lng, lat] },
        distanceField: 'distanceMeters',
        maxDistance: radiusKm * 1000,
        spherical: true,
        query,
      },
    },
    { $limit: limit },
    { $addFields: { distanceKm: { $round: [{ $divide: ['$distanceMeters', 1000] }, 2] } } },
    { $project: { distanceMeters: 0, __v: 0 } },
  ]);
  await Business.populate(results, POPULATE);

  res.json({
    success: true,
    data: results,
  });
}

```

```

    meta: { center: { lat, lng }, radiusKm, count: results.length },
  });
}

// GET /api/businesses/:id
async function getBusiness(req, res) {
  const business = await Business.findById(req.params.id).populate(POPULATE);
  if (!business) throw ApiError.notFound('Business not found');
  res.json({ success: true, data: business });
}

// POST /api/businesses (owner/admin, JSON or multipart with "photos")
async function createBusiness(req, res) {
  const category = await resolveCategory(req.body.category);
  const business = new Business({ owner: req.user._id, category: category._id });
  applyFields(business, req.body);
  await saveWithPhotos(business, req.files);
  res.status(201).json({ success: true, data: business });
}

// PUT /api/businesses/:id (owner/admin; multipart "photos" are appended)
async function updateBusiness(req, res) {
  const business = await loadOwnedBusiness(req);
  if (req.body.category) business.category = (await resolveCategory(req.body.category))._id;
  applyFields(business, req.body);
  await saveWithPhotos(business, req.files);
  res.json({ success: true, data: business });
}

// DELETE /api/businesses/:id (owner/admin) - also removes its reviews and photos
async function deleteBusiness(req, res) {
  const business = await loadOwnedBusiness(req);
  await Review.deleteMany({ business: business._id });
  await business.deleteOne();
  await Promise.all(business.photos.map(storage.deleteFile));
  res.json({ success: true, message: 'Business deleted' });
}

// POST /api/businesses/:id/photos (owner/admin)
async function addPhotos(req, res) {
  if (!req.files || req.files.length === 0) {
    throw ApiError.badRequest('Attach at least one image in the "photos" field');
  }
  const business = await loadOwnedBusiness(req);
  await saveWithPhotos(business, req.files);
  res.status(201).json({ success: true, data: business.photos });
}

// DELETE /api/businesses/:id/photos/:photoId (owner/admin)
async function deletePhoto(req, res) {
  const business = await loadOwnedBusiness(req);
  const photo = business.photos.id(req.params.photoId);
  if (!photo) throw ApiError.notFound('Photo not found');
  const removed = photo.toObject();
  photo.deleteOne();
  await business.save();
  await storage.deleteFile(removed);
  res.json({ success: true, data: business.photos });
}

module.exports = {
  listBusinesses,
  searchBusinesses,
  nearBusinesses,
  getBusiness,
  createBusiness,
  updateBusiness,
  deleteBusiness,
  addPhotos,
  deletePhoto,
};

```

src/controllers/reviewController.js

```

const mongoose = require('mongoose');
const Review = require('../models/Review');
const Business = require('../models/Business');
const ApiError = require('../utils/ApiError');
const { getPagination, buildPaginationMeta } = require('../utils/query');

const REVIEW_SORTS = {
  newest: { createdAt: -1 },

```

```

    oldest: { createdAt: 1 },
    highest: { rating: -1, createdAt: -1 },
    lowest: { rating: 1, createdAt: -1 },
  };

const POPULATE_USER = { path: 'user', select: 'name avatarUrl' };

async function loadReview(id) {
  const review = await Review.findById(id);
  if (!review) throw ApiError.notFound('Review not found');
  return review;
}

// POST /api/reviews
async function createReview(req, res) {
  const { business: businessId, rating, title, comment } = req.body;
  const business = await Business.findById(businessId).select('owner');
  if (!business) throw ApiError.notFound('Business not found');
  if (business.owner.equals(req.user._id)) {
    throw ApiError.forbidden('You cannot review your own business');
  }
  if (await Review.exists({ business: businessId, user: req.user._id })) {
    throw ApiError.conflict('You have already reviewed this business. Update your existing review instead.');
```

```

  }

  const review = await Review.create({ business: businessId, user: req.user._id, rating, title, comment });
  const stats = await Review.syncBusinessRating(businessId);
  await review.populate(POPULATE_USER);
  res.status(201).json({ success: true, data: review, businessRating: stats });
}

// GET /api/reviews/business/:id?sort=&page=&limit=
// Includes a star breakdown for the whole business, not just the current page.
async function getBusinessReviews(req, res) {
  const businessId = req.params.id;
  const business = await Business.findById(businessId).select('name averageRating reviewCount');
  if (!business) throw ApiError.notFound('Business not found');
```

```

  const { page, limit, skip } = getPagination(req.query);
  const filter = { business: businessId };
  const [reviews, total, breakdownAgg] = await Promise.all([
    Review.find(filter)
      .sort(REVIEW_SORTS[req.query.sort] || REVIEW_SORTS.newest)
      .skip(skip)
      .limit(limit)
      .populate(POPULATE_USER),
    Review.countDocuments(filter),
    Review.aggregate([
      { $match: { business: new mongoose.Types.ObjectId(businessId) } },
      { $group: { _id: '$rating', count: { $sum: 1 } } },
    ]),
  ]);

  const breakdown = { 1: 0, 2: 0, 3: 0, 4: 0, 5: 0 };
  for (const row of breakdownAgg) breakdown[row._id] = row.count;

  res.json({
    success: true,
    data: reviews,
    summary: {
      business: { _id: business._id, name: business.name },
      averageRating: business.averageRating,
      reviewCount: business.reviewCount,
      breakdown,
    },
    pagination: buildPaginationMeta(total, page, limit),
  });
}

// GET /api/reviews/me
async function getMyReviews(req, res) {
  const { page, limit, skip } = getPagination(req.query);
  const filter = { user: req.user._id };
  const [reviews, total] = await Promise.all([
    Review.find(filter).sort({ createdAt: -1 }).skip(skip).limit(limit).populate('business', 'name averageRating'),
    Review.countDocuments(filter),
  ]);
  res.json({ success: true, data: reviews, pagination: buildPaginationMeta(total, page, limit) });
}

```

```

// PUT /api/reviews/:id (author only)
async function updateReview(req, res) {
  const review = await loadReview(req.params.id);
  if (!review.user.equals(req.user._id)) throw ApiError.forbidden('You can only edit your own review');

  for (const key of ['rating', 'title', 'comment']) {
    if (req.body[key] !== undefined) review[key] = req.body[key];
  }
  await review.save();
  const stats = await Review.syncBusinessRating(review.business);
  await review.populate(POPULATE_USER);
  res.json({ success: true, data: review, businessRating: stats });
}

// DELETE /api/reviews/:id (author or admin)
async function deleteReview(req, res) {
  const review = await loadReview(req.params.id);
  if (!review.user.equals(req.user._id) && req.user.role !== 'admin') {
    throw ApiError.forbidden('You can only delete your own review');
  }
  await review.deleteOne();
  const stats = await Review.syncBusinessRating(review.business);
  res.json({ success: true, message: 'Review deleted', businessRating: stats });
}

async function loadReviewForOwner(req) {
  const review = await loadReview(req.params.id);
  const business = await Business.findById(review.business).select('owner');
  if (!business) throw ApiError.notFound('Business not found');
  if (!business.owner.equals(req.user._id) && req.user.role !== 'admin') {
    throw ApiError.forbidden('Only the business owner can respond to reviews');
  }
  return review;
}

// PUT /api/reviews/:id/response (business owner) - creates or replaces the response
async function respondToReview(req, res) {
  const review = await loadReviewForOwner(req);
  review.ownerResponse = { text: req.body.text, respondedAt: new Date() };
  await review.save();
  await review.populate(POPULATE_USER);
  res.json({ success: true, data: review });
}

// DELETE /api/reviews/:id/response (business owner)
async function deleteResponse(req, res) {
  const review = await loadReviewForOwner(req);
  if (!review.ownerResponse || !review.ownerResponse.text) {
    throw ApiError.notFound('This review has no owner response');
  }
  review.ownerResponse = undefined;
  await review.save();
  res.json({ success: true, message: 'Response removed' });
}

module.exports = {
  createReview,
  getBusinessReviews,
  getMyReviews,
  updateReview,
  deleteReview,
  respondToReview,
  deleteResponse,
};

```

src/controllers/categoryController.js

```

const Category = require('../models/Category');
const Business = require('../models/Business');
const ApiError = require('../utils/ApiError');

// GET /api/categories - each category includes how many businesses use it
async function listCategories(_req, res) {
  const [categories, counts] = await Promise.all([
    Category.find().sort({ name: 1 }).lean(),
    Business.aggregate([
      { $group: { _id: '$category', count: { $sum: 1 } } }
    ]),
  ]);
  const countById = new Map(counts.map((c) => [String(c._id), c.count]));
  const data = categories.map(({ _id, ...c }) => ({ ...c, businessCount: countById.get(String(c._id)) || 0 }));
  res.json({ success: true, data });
}

```

```

}

// POST /api/categories (admin)
async function createCategory(req, res) {
  const { name, description, icon } = req.body;
  const category = await Category.create({ name, description, icon });
  res.status(201).json({ success: true, data: category });
}

// PUT /api/categories/:id (admin)
async function updateCategory(req, res) {
  const category = await Category.findById(req.params.id);
  if (!category) throw ApiError.notFound('Category not found');
  for (const key of ['name', 'description', 'icon']) {
    if (req.body[key] !== undefined) category[key] = req.body[key];
  }
  await category.save();
  res.json({ success: true, data: category });
}

// DELETE /api/categories/:id (admin) - refused while businesses still use it
async function deleteCategory(req, res) {
  const category = await Category.findById(req.params.id);
  if (!category) throw ApiError.notFound('Category not found');
  const inUse = await Business.countDocuments({ category: category._id });
  if (inUse) throw ApiError.conflict(`Category is used by ${inUse} business(es) and cannot be deleted`);
  await category.deleteOne();
  res.json({ success: true, message: 'Category deleted' });
}

module.exports = { listCategories, createCategory, updateCategory, deleteCategory };

```

src/routes/index.js

```

const router = require('express').Router();

router.use('/auth', require('./auth'));
router.use('/businesses', require('./businesses'));
router.use('/reviews', require('./reviews'));
router.use('/categories', require('./categories'));

module.exports = router;

```

src/routes/auth.js

```

const router = require('express').Router();
const ctrl = require('../controllers/authController');
const validate = require('../middleware/validate');
const { protect } = require('../middleware/auth');
const v = require('../validators');

router.post('/register', validate(v.auth.register), ctrl.register);
router.post('/login', validate(v.auth.login), ctrl.login);
router.post('/firebase', validate(v.auth.firebase), ctrl.firebaseLogin);
router.get('/me', protect, ctrl.me);

module.exports = router;

```

src/routes/businesses.js

```

const router = require('express').Router();
const ctrl = require('../controllers/businessController');
const validate = require('../middleware/validate');
const { protect, optionalAuth, authorize } = require('../middleware/auth');
const { uploadPhotos } = require('../middleware/upload');
const v = require('../validators');

const ownerOnly = [protect, authorize('owner', 'admin')];

// Static paths must come before /:id
router.get('/search', optionalAuth, validate(v.business.search), ctrl.searchBusinesses);
router.get('/near', validate(v.business.near), ctrl.nearBusinesses);

router.get('/', optionalAuth, validate(v.business.list), ctrl.listBusinesses);
// multer runs before validation so multipart text fields are available on req.body
router.post('/', ...ownerOnly, uploadPhotos, validate(v.business.create), ctrl.createBusiness);

router.get('/:id', validate(v.business.id), ctrl.getBusiness);
router.put('/:id', ...ownerOnly, uploadPhotos, validate(v.business.update), ctrl.updateBusiness);
router.delete('/:id', ...ownerOnly, validate(v.business.id), ctrl.deleteBusiness);

```

```

router.post('/:id/photos', ...ownerOnly, uploadPhotos, validate(v.business.id), ctrl.addPhotos);
router.delete('/:id/photos/:photoId', ...ownerOnly, validate(v.business.photo), ctrl.deletePhoto);

module.exports = router;

```

src/routes/reviews.js

```

const router = require('express').Router();
const ctrl = require('../controllers/reviewController');
const validate = require('../middleware/validate');
const { protect, authorize } = require('../middleware/auth');
const v = require('../validators');

router.post('/', protect, validate(v.review.create), ctrl.createReview);
router.get('/me', protect, ctrl.getMyReviews);
router.get('/business/:id', validate(v.review.forBusiness), ctrl.getBusinessReviews);

router.put('/:id', protect, validate(v.review.update), ctrl.updateReview);
router.delete('/:id', protect, validate(v.review.id), ctrl.deleteReview);

// Business owner response to a review (POST and PUT both upsert)
const ownerOnly = [protect, authorize('owner', 'admin')];
router.post('/:id/response', ...ownerOnly, validate(v.review.respond), ctrl.respondToReview);
router.put('/:id/response', ...ownerOnly, validate(v.review.respond), ctrl.respondToReview);
router.delete('/:id/response', ...ownerOnly, validate(v.review.id), ctrl.deleteResponse);

module.exports = router;

```

src/routes/categories.js

```

const router = require('express').Router();
const ctrl = require('../controllers/categoryController');
const validate = require('../middleware/validate');
const { protect, authorize } = require('../middleware/auth');
const v = require('../validators');

const adminOnly = [protect, authorize('admin')];

router.get('/', ctrl.listCategories);
router.post('/', ...adminOnly, validate(v.category.create), ctrl.createCategory);
router.put('/:id', ...adminOnly, validate(v.category.update), ctrl.updateCategory);
router.delete('/:id', ...adminOnly, validate(v.category.id), ctrl.deleteCategory);

module.exports = router;

```

src/services/storage.js

```

const fs = require('fs/promises');
const path = require('path');
const crypto = require('crypto');
const env = require('../config/env');
const { getFirebaseApp, isStorageEnabled } = require('../config/firebase');
const ApiError = require('../utils/ApiError');

const LOCAL_DIR = path.join(__dirname, '..', '..', 'uploads');

const EXT_BY_TYPE = {
  'image/jpeg': '.jpg',
  'image/png': '.png',
  'image/webp': '.webp',
  'image/gif': '.gif',
};

function buildKey(folder, file) {
  const ext = EXT_BY_TYPE[file.mimetype] || path.extname(file.originalname) || '';
  return `${folder}/${Date.now()}-${crypto.randomBytes(6).toString('hex')}${ext}`;
}

async function uploadToFirebase(file, key) {
  const bucket = getFirebaseApp().storage().bucket();
  const token = crypto.randomUUID();
  await bucket.file(key).save(file.buffer, {
    resumable: false,
    metadata: {
      contentType: file.mimetype,
      // Download token gives a stable public URL without making the bucket public
      metadata: { firebaseStorageDownloadTokens: token },
    },
  });
}

```

```

    const url = `https://firebasestorage.googleapis.com/v0/b/${bucket.name}/o/${encodeURIComponent(key)}?
alt=media&token=${token}`;
    return { url, path: key, provider: 'firebase' };
}

async function uploadToLocal(file, key) {
    const dest = path.join(LOCAL_DIR, key);
    await fs.mkdir(path.dirname(dest), { recursive: true });
    await fs.writeFile(dest, file.buffer);
    return { url: `${env.baseUrl}/uploads/${key}`, path: key, provider: 'local' };
}

// 'firebase', 'local', or 'unavailable' (serverless without Firebase)
function storageMode() {
    if (isStorageEnabled()) return 'firebase';
    return env.isServerless ? 'unavailable' : 'local';
}

async function uploadFile(file, folder) {
    const key = buildKey(folder, file);
    const mode = storageMode();
    if (mode === 'unavailable') {
        throw new ApiError(503, 'Photo uploads need Firebase Storage configured on this deployment');
    }
    return mode === 'firebase' ? uploadToFirebase(file, key) : uploadToLocal(file, key);
}

function uploadFiles(files = [], folder) {
    return Promise.all(files.map(f => uploadFile(f, folder)));
}

// Best effort: a missing file should not fail the request that removes it.
async function deleteFile(photo) {
    try {
        if (photo.provider === 'firebase' && isStorageEnabled()) {
            await getFirebaseApp().storage().bucket().file(photo.path).delete({ ignoreNotFound: true });
        } else if (photo.provider === 'local') {
            const target = path.resolve(LOCAL_DIR, photo.path);
            if (target.startsWith(LOCAL_DIR + path.sep)) await fs.rm(target, { force: true });
        }
    } catch (err) {
        console.warn(`[storage] could not delete ${photo.path}: ${err.message}`);
    }
}

module.exports = { uploadFile, uploadFiles, deleteFile, storageMode, LOCAL_DIR };

```

src/utils/ApiError.js

```

class ApiError extends Error {
    constructor(statusCode, message, details) {
        super(message);
        this.statusCode = statusCode;
        this.details = details;
    }

    static badRequest(msg, details) { return new ApiError(400, msg, details); }
    static unauthorized(msg = 'Authentication required') { return new ApiError(401, msg); }
    static forbidden(msg = 'You do not have permission to perform this action') { return new ApiError(403, msg); }
    static notFound(msg = 'Resource not found') { return new ApiError(404, msg); }
    static conflict(msg) { return new ApiError(409, msg); }
}

module.exports = ApiError;

```

src/utils/query.js

```

// Pure helpers for list/search endpoints. No DB access, so they're unit-testable.

const MAX_LIMIT = 50;

function escapeRegex(str) {
    return String(str).replace(/[\.\*\+\?\^\$\{\}\|\[\]\|\]/g, '\\$&');
}

function getPagination(query = {}, defaultLimit = 10) {
    const page = Math.max(1, parseInt(query.page, 10) || 1);
    const limit = Math.min(MAX_LIMIT, Math.max(1, parseInt(query.limit, 10) || defaultLimit));
    return { page, limit, skip: (page - 1) * limit };
}

```

```

function buildPaginationMeta(total, page, limit) {
  return { total, page, limit, pages: Math.ceil(total / limit) || 0 };
}

const SORTS = {
  rating: { averageRating: -1, reviewCount: -1 },
  reviews: { reviewCount: -1 },
  newest: { createdAt: -1 },
  oldest: { createdAt: 1 },
  name: { name: 1 },
};

function getSort(sortKey) {
  return SORTS[sortKey] || SORTS.newest;
}

function slugify(str) {
  return String(str)
    .normalize('NFD')
    .replace(/[\u0300-\u036f]/g, '')
    .toLowerCase()
    .trim()
    .replace(/^[a-z0-9\s-]/g, '')
    .replace(/[\s_-]+/g, '-')
    .replace(/^-+|-+$/g, '');
}

// Turns an { avg, count } aggregation result into the cached business fields,
// rounding to one decimal. Missing result (no reviews) means zero.
function roundRatingStats(agg) {
  if (!agg || !agg.count) return { averageRating: 0, reviewCount: 0 };
  return { averageRating: Math.round(agg.avg * 10) / 10, reviewCount: agg.count };
}

module.exports = {
  MAX_LIMIT,
  escapeRegex,
  getPagination,
  buildPaginationMeta,
  getSort,
  slugify,
  roundRatingStats,
};

```

src/docs/swaggerPage.js

```

// Swagger UI loaded from a CDN. swagger-ui-express serves its assets from
// node_modules, which Vercel's function bundles don't include.
const router = require('express').Router();

const CDN = 'https://cdn.jsdelivr.net/npm/swagger-ui-dist@5';

const html = `<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>TownRate API Docs</title>
  <link rel="stylesheet" href="${CDN}/swagger-ui.css">
</head>
<body>
  <div id="swagger-ui"></div>
  <script src="${CDN}/swagger-ui-bundle.js"></script>
  <script src="/api-docs/init.js"></script>
</body>
</html>`;

// Separate file rather than an inline script so the default CSP can stay strict
const initJs = `window.ui = SwaggerUIBundle({
  url: '/api-docs.json',
  dom_id: '#swagger-ui',
  persistAuthorization: true,
});`;

router.get('/init.js', (_req, res) => res.type('application/javascript').send(initJs));
router.get('/', (_req, res) => res.type('html').send(html));

module.exports = router;

```

src/docs/openapi.js

```
// OpenAPI 3 spec served at /api-docs (Swagger UI) and /api-docs.json.

const bearer = [{ bearerAuth: [] }];
const ref = (name) => ({ $ref: `#/components/schemas/${name}` });
const json = (schema) => ({ 'application/json': { schema } });
const idParam = (name = 'id', description = 'MongoDB ObjectId') => ({
  name, in: 'path', required: true, description, schema: { type: 'string' },
});
const q = (name, description, schema = { type: 'string' }, required = false) => ({
  name, in: 'query', required, description, schema,
});
const pageParams = [
  q('page', 'Page number', { type: 'integer', minimum: 1, default: 1 }),
  q('limit', 'Items per page (max 50)', { type: 'integer', minimum: 1, maximum: 50, default: 10 }),
];
const ok = (description, schema) => ({ description, content: schema ? json(schema) : undefined });
const err = (description) => ({ description, content: json(ref('Error')) });

const businessFields = {
  name: { type: 'string', example: 'Blue Tokai Coffee' },
  description: { type: 'string', example: 'Specialty coffee roasters' },
  category: { type: 'string', description: 'Category id, slug or name', example: 'cafe' },
  tags: { type: 'array', items: { type: 'string' }, example: ['coffee', 'wifi'] },
  address: {
    type: 'object',
    properties: {
      street: { type: 'string' }, city: { type: 'string', example: 'Mumbai' },
      state: { type: 'string' }, zip: { type: 'string' }, country: { type: 'string' },
    },
  },
  latitude: { type: 'number', example: 19.076 },
  longitude: { type: 'number', example: 72.8777 },
  phone: { type: 'string', example: '+91 98765 43210' },
  email: { type: 'string', format: 'email' },
  website: { type: 'string', format: 'uri' },
};

const businessBody = (required) => ({
  required: true,
  content: {
    'application/json': { schema: { type: 'object', required, properties: businessFields } },
    'multipart/form-data': {
      schema: {
        type: 'object',
        required,
        properties: {
          ...businessFields,
          'address[city]': { type: 'string', description: 'Use bracket notation for address fields in multipart' },
          tags: { type: 'string', description: 'Comma-separated in multipart', example: 'coffee,wifi' },
          photos: { type: 'array', items: { type: 'string', format: 'binary' }, description: 'Up to 5 images (jpeg/png/webp/gif)' },
        },
      },
    },
  },
});

module.exports = {
  openapi: '3.0.3',
  info: {
    title: 'TownRate API',
    version: '1.0.0',
    description:
      'Local Business Review Platform. Authenticate with `POST /api/auth/login` (or `/api/auth/firebase`), ' +
      'then click **Authorize** and paste the token.\n\nRoles: `user` (reviews), `owner` (businesses, review responses), `admin` (everything, categories).',
  },
  servers: [{ url: '/', description: 'This server' }],
  tags: [
    { name: 'Auth' }, { name: 'Businesses' }, { name: 'Photos' },
    { name: 'Reviews' }, { name: 'Categories' }, { name: 'System' },
  ],
  components: {
    securitySchemes: { bearerAuth: { type: 'http', scheme: 'bearer', bearerFormat: 'JWT' } },
    schemas: {
      Error: {
        type: 'object',
        properties: {
          success: { type: 'boolean', example: false },
          message: { type: 'string' },
        },
      },
    },
  },
};
```

```

      errors: { type: 'array', items: { type: 'object', properties: { field: { type: 'string' } },
message: { type: 'string' } } } },
    },
    },
    Pagination: {
      type: 'object',
      properties: { total: { type: 'integer' }, page: { type: 'integer' }, limit: { type: 'integer' },
pages: { type: 'integer' } },
    },
    User: {
      type: 'object',
      properties: {
        _id: { type: 'string' }, name: { type: 'string' }, email: { type: 'string' },
        role: { type: 'string', enum: ['user', 'owner', 'admin'] }, avatarUrl: { type: 'string' },
        createdAt: { type: 'string', format: 'date-time' },
      },
    },
    AuthResult: {
      type: 'object',
      properties: {
        success: { type: 'boolean' },
        data: { type: 'object', properties: { token: { type: 'string' }, user: ref('User') } },
      },
    },
    Category: {
      type: 'object',
      properties: {
        _id: { type: 'string' }, name: { type: 'string' }, slug: { type: 'string' },
        description: { type: 'string' }, icon: { type: 'string' }, businessCount: { type: 'integer' },
      },
    },
    Photo: {
      type: 'object',
      properties: {
        _id: { type: 'string' }, url: { type: 'string' }, path: { type: 'string' },
        provider: { type: 'string', enum: ['firebase', 'local'] },
      },
    },
    Business: {
      type: 'object',
      properties: {
        _id: { type: 'string' },
        owner: { type: 'object', properties: { _id: { type: 'string' }, name: { type: 'string' } } },
        name: { type: 'string' }, description: { type: 'string' },
        category: { type: 'object', properties: { _id: { type: 'string' }, name: { type: 'string' } },
slug: { type: 'string' } } },
        tags: { type: 'array', items: { type: 'string' } },
        address: businessFields.address,
        location: {
          type: 'object',
          properties: { type: { type: 'string', example: 'Point' }, coordinates: { type: 'array', items:
{ type: 'number' }, example: [72.8777, 19.076] } } },
        },
        phone: { type: 'string' }, email: { type: 'string' }, website: { type: 'string' },
        photos: { type: 'array', items: ref('Photo') },
        averageRating: { type: 'number', example: 4.5 }, reviewCount: { type: 'integer' },
        distanceKm: { type: 'number', description: 'Only on /near results' },
      },
    },
    Review: {
      type: 'object',
      properties: {
        _id: { type: 'string' }, business: { type: 'string' },
        user: { type: 'object', properties: { _id: { type: 'string' }, name: { type: 'string' } } },
        rating: { type: 'integer', minimum: 1, maximum: 5 }, title: { type: 'string' }, comment: { type:
'string' },
        ownerResponse: { type: 'object', properties: { text: { type: 'string' }, respondedAt: { type:
'string', format: 'date-time' } } },
        createdAt: { type: 'string', format: 'date-time' },
      },
    },
    },
    paths: {
      '/api/health': {
        get: { tags: ['System'], summary: 'Health check (DB, Firebase status)', responses: { 200:
ok('Healthy'), 503: ok('Database unavailable') } } },
    },
    '/api/auth/register': {
      post: {
        tags: ['Auth'], summary: 'Register with email and password',
        requestBody: {
          required: true,
          content: json({
            type: 'object', required: ['name', 'email', 'password'],
            properties: {

```

```

        name: { type: 'string', example: 'Priya Sharma' }, email: { type: 'string', example:
'priya@example.com' },
        password: { type: 'string', minLength: 6, example: 'secret123' },
        role: { type: 'string', enum: ['user', 'owner'], default: 'user' },
    },
    },
    },
    responses: { 201: ok('Registered', ref('AuthResult')), 409: err('Email taken'), 422:
err('Validation failed') },
    },
    },
    '/api/auth/login': {
    post: {
        tags: ['Auth'], summary: 'Log in, returns a JWT',
        requestBody: {
            required: true,
            content: json({ type: 'object', required: ['email', 'password'], properties: { email: { type:
'string', example: 'priya@example.com' }, password: { type: 'string', example: 'secret123' } } })),
        },
        responses: { 200: ok('Logged in', ref('AuthResult')), 401: err('Invalid credentials') },
    },
    },
    '/api/auth/firebase': {
    post: {
        tags: ['Auth'], summary: 'Exchange a Firebase ID token for an app JWT',
        description: 'Verifies the token with Firebase Admin, links to an existing account by email or
creates one.',
        requestBody: {
            required: true,
            content: json({ type: 'object', required: ['idToken'], properties: { idToken: { type: 'string' },
role: { type: 'string', enum: ['user', 'owner'] } } })),
        },
        responses: { 200: ok('Logged in', ref('AuthResult')), 201: ok('Account created',
ref('AuthResult')), 401: err('Bad token'), 503: err('Firebase not configured') },
    },
    },
    '/api/auth/me': {
    get: { tags: ['Auth'], summary: 'Current user', security: bearer, responses: { 200: ok('User',
{ type: 'object', properties: { data: ref('User') } } ), 401: err('Unauthenticated') } },
    },
    },
    '/api/businesses': {
    get: {
        tags: ['Businesses'], summary: 'List businesses (filter by category / location)',
        parameters: [
            q('category', 'Category id, slug or name'), q('city', 'City (case-insensitive exact match)'),
            q('minRating', 'Minimum average rating', { type: 'number', minimum: 0, maximum: 5 }),
            q('owner', 'Owner id, or "me" (requires token)'),
            q('sort', 'Sort order', { type: 'string', enum: ['newest', 'oldest', 'rating', 'reviews', 'name']
}),
        ],
        ...pageParams,
    },
    responses: { 200: ok('Businesses', { type: 'object', properties: { data: { type: 'array', items:
ref('Business') }, pagination: ref('Pagination') } } ) },
    },
    post: {
        tags: ['Businesses'], summary: 'Create a business profile (owner/admin)', security: bearer,
        requestBody: businessBody(['name', 'category', 'address']),
        responses: { 201: ok('Created', { type: 'object', properties: { data: ref('Business') } } ), 401:
err('Unauthenticated'), 403: err('Not an owner'), 422: err('Validation failed') },
    },
    },
    '/api/businesses/search': {
    get: {
        tags: ['Businesses'], summary: 'Search businesses by keyword',
        description: 'Case-insensitive partial match on name, description, tags, city and category name. At
least one of keyword, category or city is required. Sorted by rating by default.',
        parameters: [
            q('keyword', 'Search text', { type: 'string', example: 'cafe' }), q('category', 'Category id,
slug or name'),
            q('city', 'City'), q('minRating', 'Minimum rating', { type: 'number' }),
            q('sort', 'Sort order', { type: 'string', enum: ['rating', 'reviews', 'newest', 'oldest', 'name']
}),
        ],
        ...pageParams,
    },
    responses: { 200: ok('Results', { type: 'object', properties: { data: { type: 'array', items:
ref('Business') }, pagination: ref('Pagination') } } ), 422: err('No search criteria') },
    },
    },
    '/api/businesses/near': {
    get: {
        tags: ['Businesses'], summary: '"Near me" geo search', description: 'Businesses within `radius` km
of the point, nearest first, each with `distanceKm`.',
        parameters: [
            q('lat', 'Latitude', { type: 'number', example: 19.076 }, true), q('lng', 'Longitude', { type:
'number', example: 72.8777 }, true),
            q('radius', 'Radius in km (0.1-100)', { type: 'number', default: 5 }), q('category', 'Category
id, slug or name'),

```

```

        q('minRating', 'Minimum rating', { type: 'number' }), q('limit', 'Max results (max 50)', { type:
'integer', default: 20 })),
    },
    responses: { 200: ok('Nearby businesses', { type: 'object', properties: { data: { type: 'array',
items: ref('Business') } } })), 422: err('Missing coordinates') },
    },
    '/api/businesses/{id}': {
      get: { tags: ['Businesses'], summary: 'Get a business', parameters: [idParam()], responses: { 200:
ok('Business', { type: 'object', properties: { data: ref('Business') } })), 404: err('Not found') } },
      put: {
        tags: ['Businesses'], summary: 'Update a business (its owner or admin)', security: bearer,
parameters: [idParam()],
        description: 'Partial update. Photos sent as multipart are appended.',
        requestBody: businessBody([]),
        responses: { 200: ok('Updated', { type: 'object', properties: { data: ref('Business') } })), 403:
err('Not the owner'), 404: err('Not found') },
      },
      delete: {
        tags: ['Businesses'], summary: 'Delete a business with its reviews and photos (owner/admin)',
security: bearer, parameters: [idParam()],
        responses: { 200: ok('Deleted'), 403: err('Not the owner'), 404: err('Not found') },
      },
    },
    '/api/businesses/{id}/photos': {
      post: {
        tags: ['Photos'], summary: 'Upload photos (Firebase Storage)', security: bearer, parameters:
[idParam()],
        requestBody: { required: true, content: { 'multipart/form-data': { schema: { type: 'object',
properties: { photos: { type: 'array', items: { type: 'string', format: 'binary' } } } } } },
        responses: { 201: ok('All photos of the business', { type: 'object', properties: { data: { type:
'array', items: ref('Photo') } } })), 400: err('No file / bad type / too large') },
      },
    },
    '/api/businesses/{id}/photos/{photoId}': {
      delete: {
        tags: ['Photos'], summary: 'Delete a photo', security: bearer, parameters: [idParam(),
idParam('photoId', 'Photo id')],
        responses: { 200: ok('Remaining photos'), 404: err('Not found') },
      },
    },
    '/api/reviews': {
      post: {
        tags: ['Reviews'], summary: 'Leave a review (one per user per business)', security: bearer,
requestBody: {
          required: true,
          content: json({
            type: 'object', required: ['business', 'rating'],
            properties: {
              business: { type: 'string' }, rating: { type: 'integer', minimum: 1, maximum: 5, example:
5 },
              title: { type: 'string', example: 'Great coffee' }, comment: { type: 'string', example:
'Loved the cold brew.' },
            },
          }),
        responses: { 201: ok('Created', { type: 'object', properties: { data: ref('Review') } })), 403:
err('Own business'), 404: err('Business not found'), 409: err('Already reviewed') },
      },
    },
    '/api/reviews/me': {
      get: { tags: ['Reviews'], summary: "Current user's reviews", security: bearer, parameters:
pageParams, responses: { 200: ok('Reviews') } },
    },
    '/api/reviews/business/{id}': {
      get: {
        tags: ['Reviews'], summary: 'All reviews for a business, with star breakdown', parameters: [
idParam('id', 'Business id'), q('sort', 'Sort order', { type: 'string', enum: ['newest',
'oldest', 'highest', 'lowest'] })), ...pageParams,
        responses: { 200: ok('Reviews', { type: 'object', properties: { data: { type: 'array', items:
ref('Review') }, summary: { type: 'object' }, pagination: ref('Pagination') } })), 404: err('Business not
found') },
      },
    },
    '/api/reviews/{id}': {
      put: {
        tags: ['Reviews'], summary: 'Edit your review', security: bearer, parameters: [idParam()],
requestBody: { content: json({ type: 'object', properties: { rating: { type: 'integer' }, title:
{ type: 'string' }, comment: { type: 'string' } } })),
        responses: { 200: ok('Updated'), 403: err('Not the author'), 404: err('Not found') },
      },
      delete: { tags: ['Reviews'], summary: 'Delete your review (or any, as admin)', security: bearer,
parameters: [idParam()], responses: { 200: ok('Deleted'), 403: err('Not the author') } },
    },
    '/api/reviews/{id}/response': {

```

```

    post: {
      tags: ['Reviews'], summary: 'Business owner responds to a review (POST or PUT, upserts)', security:
bearer, parameters: [idParam()],
      requestBody: { required: true, content: json({ type: 'object', required: ['text'], properties:
{ text: { type: 'string', example: 'Thanks for visiting!' } } ) } },
      responses: { 200: ok('Review with response', { type: 'object', properties: { data:
ref('Review') } } ), 403: err('Not the business owner') },
    },
    put: {
      tags: ['Reviews'], summary: 'Same as POST', security: bearer, parameters: [idParam()],
      requestBody: { required: true, content: json({ type: 'object', required: ['text'], properties:
{ text: { type: 'string' } } ) } },
      responses: { 200: ok('Review with response') },
    },
    delete: { tags: ['Reviews'], summary: 'Remove the owner response', security: bearer, parameters:
[idParam()], responses: { 200: ok('Removed'), 404: err('No response') } },
  },

  '/api/categories': {
    get: { tags: ['Categories'], summary: 'List categories with business counts', responses: { 200:
ok('Categories', { type: 'object', properties: { data: { type: 'array', items: ref('Category') } } } ) } },
    post: {
      tags: ['Categories'], summary: 'Create category (admin)', security: bearer,
      requestBody: { required: true, content: json({ type: 'object', required: ['name'], properties:
{ name: { type: 'string' }, description: { type: 'string' }, icon: { type: 'string' } } ) } },
      responses: { 201: ok('Created'), 403: err('Admin only'), 409: err('Duplicate') },
    },
  },
  '/api/categories/{id}': {
    put: {
      tags: ['Categories'], summary: 'Update category (admin)', security: bearer, parameters:
[idParam()],
      requestBody: { content: json({ type: 'object', properties: { name: { type: 'string' }, description:
{ type: 'string' }, icon: { type: 'string' } } ) } },
      responses: { 200: ok('Updated') },
    },
    delete: { tags: ['Categories'], summary: 'Delete unused category (admin)', security: bearer,
parameters: [idParam()], responses: { 200: ok('Deleted'), 409: err('In use') } },
  },
},
};

```

scripts/seed.js

```

// Seeds categories, demo users, businesses and reviews.
// Usage: npm run seed (adds categories only if missing, skips if demo data exists)
// npm run seed -- --fresh (wipes the database first)
// npm run seed -- --categories-only (production: no demo accounts)
const mongoose = require('mongoose');
const { connectDB, disconnectDB } = require('../src/config/db');
const User = require('../src/models/User');
const Category = require('../src/models/Category');
const Business = require('../src/models/Business');
const Review = require('../src/models/Review');

const CATEGORIES = [
  { name: 'Cafe', icon: '☕', description: 'Coffee shops and cafes' },
  { name: 'Restaurant', icon: '🍴', description: 'Dining and eateries' },
  { name: 'Bakery', icon: '🍰', description: 'Breads, cakes and pastries' },
  { name: 'Salon', icon: '💇', description: 'Hair and beauty' },
  { name: 'Gym', icon: '💪', description: 'Fitness centres' },
  { name: 'Grocery', icon: '🛒', description: 'Supermarkets and kirana stores' },
  { name: 'Pharmacy', icon: '💊', description: 'Chemists and medical stores' },
  { name: 'Bookstore', icon: '📖', description: 'Books and stationery' },
];

const PASSWORD = 'password123';

async function main() {
  await connectDB();

  if (process.argv.includes('--fresh')) {
    await mongoose.connection.dropDatabase();
    console.log('[seed] database dropped');
  }

  for (const c of CATEGORIES) {
    await Category.updateOne({ name: c.name }, { $setOnInsert: c }, { upsert: true });
  }
  // updateOne bypasses the slug hook, so fill any missing slugs
  for (const c of await Category.find({ slug: { $exists: false } })) await c.save();
  console.log(`[seed] ${CATEGORIES.length} categories ready`);
  if (process.argv.includes('--categories-only')) return;
}

```

```

if (await User.exists({ email: 'admin@townrate.dev' })) {
  console.log('[seed] demo data already present (use --fresh to reset)');
  return;
}

const [admin, owner, owner2, alice, bob] = await User.create([
  { name: 'Admin', email: 'admin@townrate.dev', password: PASSWORD, role: 'admin' },
  { name: 'Rahul Mehta', email: 'owner@townrate.dev', password: PASSWORD, role: 'owner' },
  { name: 'Sara Khan', email: 'owner2@townrate.dev', password: PASSWORD, role: 'owner' },
  { name: 'Alice', email: 'alice@townrate.dev', password: PASSWORD, role: 'user' },
  { name: 'Bob', email: 'bob@townrate.dev', password: PASSWORD, role: 'user' },
]);

const cat = Object.fromEntries((await Category.find()).map((c) => [c.slug, c._id]));
const point = (lat, lng) => ({ type: 'Point', coordinates: [lng, lat] });

const businesses = await Business.create([
  {
    owner: owner._id, name: 'Bean There Cafe', category: cat.cafe, tags: ['coffee', 'wifi', 'breakfast'],
    description: 'Cozy neighbourhood cafe with single-origin pour-overs.',
    address: { street: '12 Hill Road', city: 'Mumbai', state: 'Maharashtra', zip: '400050' },
    location: point(19.0544, 72.8406), phone: '+91 98200 11111',
  },
  {
    owner: owner._id, name: 'Sunrise Bakery', category: cat.bakery, tags: ['bread', 'cakes'],
    description: 'Fresh sourdough every morning.',
    address: { street: '4 Carter Road', city: 'Mumbai', state: 'Maharashtra', zip: '400050' },
    location: point(19.0650, 72.8225),
  },
  {
    owner: owner2._id, name: 'Chai & Chapters', category: cat.cafe, tags: ['tea', 'books', 'quiet'],
    description: 'A reading cafe serving masala chai and snacks.',
    address: { street: '88 Linking Road', city: 'Mumbai', state: 'Maharashtra', zip: '400054' },
    location: point(19.0728, 72.8347),
  },
  {
    owner: owner2._id, name: 'Iron Temple Gym', category: cat.gym, tags: ['weights', 'cardio'],
    description: 'Open 24/7 with certified trainers.',
    address: { street: '5 MG Road', city: 'Pune', state: 'Maharashtra', zip: '411001' },
    location: point(18.5196, 73.8553),
  },
  {
    owner: owner._id, name: 'Spice Route Kitchen', category: cat.restaurant, tags: ['north indian', 'family'],
    description: 'Tandoor specialities and thalis.',
    address: { street: '21 Brigade Road', city: 'Bengaluru', state: 'Karnataka', zip: '560025' },
    location: point(12.9716, 77.6070),
  },
]);

const [beanThere, sunrise, chai, gym] = businesses;
await Review.create([
  { business: beanThere._id, user: alice._id, rating: 5, title: 'Best cold brew', comment: 'Friendly staff and fast wifi.' },
  { business: beanThere._id, user: bob._id, rating: 4, title: 'Good but busy', comment: 'Hard to find a seat on weekends.' },
  { business: sunrise._id, user: alice._id, rating: 4, comment: 'Sourdough is excellent.' },
  { business: chai._id, user: bob._id, rating: 5, comment: 'Perfect spot to read.' },
  { business: gym._id, user: alice._id, rating: 3, comment: 'Good equipment, crowded evenings.' },
]);
await Review.updateOne(
  { business: beanThere._id, user: bob._id },
  { ownerResponse: { text: 'Thanks Bob! We added more seating last week.', respondedAt: new Date() } }
);
for (const b of businesses) await Review.syncBusinessRating(b._id);

console.log(`[seed] created 5 users (password "${PASSWORD}"), ${businesses.length} businesses, 5 reviews`);
console.log('[seed] logins: admin@townrate.dev, owner@townrate.dev, owner2@townrate.dev, alice@townrate.dev, bob@townrate.dev');
}

main()
  .catch((err) => {
    console.error('[seed] failed:', err);
    process.exitCode = 1;
  })
  .finally(() => disconnectDB());

```

tests/query.test.js

```
const test = require('node:test');
```

```

const assert = require('node:assert/strict');
const {
  escapeRegex, getPagination, buildPaginationMeta, getSort, slugify, roundRatingStats, MAX_LIMIT,
} = require('../src/utils/query');

test('escapeRegex neutralises regex metacharacters', () => {
  const rx = new RegExp(escapeRegex('caf.* (best)?'), 'i');
  assert.ok(rx.test('Caf.* (Best)? place'));
  assert.ok(!rx.test('cafe best'));
});

test('getPagination defaults, clamps and computes skip', () => {
  assert.deepEqual(getPagination({}), { page: 1, limit: 10, skip: 0 });
  assert.deepEqual(getPagination({ page: '3', limit: '5' }), { page: 3, limit: 5, skip: 10 });
  assert.equal(getPagination({ limit: '1000' }).limit, MAX_LIMIT);
  assert.deepEqual(getPagination({ page: '-2', limit: 'abc' }), { page: 1, limit: 10, skip: 0 });
});

test('buildPaginationMeta computes page count', () => {
  assert.deepEqual(buildPaginationMeta(21, 1, 10), { total: 21, page: 1, limit: 10, pages: 3 });
  assert.equal(buildPaginationMeta(0, 1, 10).pages, 0);
});

test('getSort falls back to newest', () => {
  assert.deepEqual(getSort('rating'), { averageRating: -1, reviewCount: -1 });
  assert.deepEqual(getSort('bogus'), { createdAt: -1 });
});

test('slugify', () => {
  assert.equal(slugify(' Coffee & Tea Shops '), 'coffee-tea-shops');
  assert.equal(slugify('Café Déjà Vu'), 'cafe-deja-vu');
});

test('roundRatingStats rounds to one decimal and handles no reviews', () => {
  assert.deepEqual(roundRatingStats(undefined), { averageRating: 0, reviewCount: 0 });
  assert.deepEqual(roundRatingStats({ avg: 4.333333, count: 3 }), { averageRating: 4.3, reviewCount: 3 });
  assert.deepEqual(roundRatingStats({ avg: 4.25, count: 4 }), { averageRating: 4.3, reviewCount: 4 });
});

```

tests/api.test.js

```

// End-to-end API tests against a real MongoDB.
// Uses TEST_MONGO_URI (default mongodb://127.0.0.1:27017/townrate_test); skipped if unreachable.
process.env.NODE_ENV = 'test';
process.env.JWT_SECRET = 'test-secret';
// Force local photo storage even if the developer's .env has Firebase credentials
for (const k of ['FIREBASE_PROJECT_ID', 'FIREBASE_CLIENT_EMAIL', 'FIREBASE_PRIVATE_KEY',
'GOOGLE_APPLICATION_CREDENTIALS']) {
  process.env[k] = '';
}

const test = require('node:test');
const assert = require('node:assert/strict');
const fs = require('node:fs');
const path = require('node:path');
const mongoose = require('mongoose');
const request = require('supertest');

const app = require('../src/app');
const Category = require('../src/models/Category');
const User = require('../src/models/User');
const { LOCAL_DIR } = require('../src/services/storage');

const MONGO_URI = process.env.TEST_MONGO_URI || 'mongodb://127.0.0.1:27017/townrate_test';

// 1x1 transparent PNG
const PNG = Buffer.from(
  'iVBORw0KGgoAAAANSUhEUgAAAAEAAAABCAQAAAFcSJAADADU1EQVR42mNkYPhfDwAChwGA60e6kgAAABJRU5ErkJggg==',
  'base64'
);

let dbAvailable = true;
const ctx = {};

test.before(async () => {
  try {
    await mongoose.connect(MONGO_URI, { serverSelectionTimeoutMS: 2000 });
    await mongoose.connection.dropDatabase();
    await Promise.all(mongoose.modelNames().map((m) => mongoose.model(m).syncIndexes()));
  } catch (err) {

```

```

    dbAvailable = false;
    console.warn(`[test] MongoDB not reachable at ${MONGO_URI}, skipping API tests (${err.message})`);
  }
});

test.after(async () => {
  if (dbAvailable) {
    await mongoose.connection.dropDatabase();
    await mongoose.disconnect();
  }
  if (ctx.businessId) fs.rmSync(path.join(LOCAL_DIR, 'businesses', ctx.businessId), { recursive: true, force: true });
});

const api = () => request(app);
const auth = (token) => ({ Authorization: `Bearer ${token}` });

test('TownRate API', async (t) => {
  if (!dbAvailable) return t.skip('MongoDB unavailable');

  await t.test('health reports database connected', async () => {
    const res = await api().get('/api/health').expect(200);
    assert.equal(res.body.database, 'connected');
    assert.equal(res.body.photoStorage, 'local');
  });

  await t.test('setup: categories and admin', async () => {
    await Category.create([
      { name: 'Cafe' },
      { name: 'Gym' }
    ]);
    await User.create({
      name: 'Admin',
      email: 'admin@test.dev',
      password: 'secret123',
      role: 'admin'
    });
    const res = await api().post('/api/auth/login').send({
      email: 'admin@test.dev',
      password: 'secret123'
    }).expect(200);
    ctx.adminToken = res.body.data.token;
  });

  await t.test('POST /api/auth/register validates input', async () => {
    const res = await api().post('/api/auth/register').send({
      name: '',
      email: 'nope',
      password: '1'
    }).expect(422);
    const fields = res.body.errors.map((e) => e.field);
    assert.ok(fields.includes('name') && fields.includes('email') && fields.includes('password'));
  });

  await t.test('POST /api/auth/register refuses admin self-registration', async () => {
    await api().post('/api/auth/register').send({
      name: 'X',
      email: 'x@test.dev',
      password: 'secret123',
      role: 'admin'
    }).expect(422);
  });

  await t.test('register owner and two users', async () => {
    const owner = await api().post('/api/auth/register').send({
      name: 'Owner',
      email: 'owner@test.dev',
      password: 'secret123',
      role: 'owner'
    }).expect(201);
    ctx.ownerToken = owner.body.data.token;
    assert.equal(owner.body.data.user.role, 'owner');
    assert.equal(owner.body.data.user.password, undefined);

    const u1 = await api().post('/api/auth/register').send({
      name: 'Alice',
      email: 'Alice@Test.dev',
      password: 'secret123'
    }).expect(201);
    ctx.aliceToken = u1.body.data.token;
    assert.equal(u1.body.data.user.email, 'alice@test.dev');
    const u2 = await api().post('/api/auth/register').send({
      name: 'Bob',
      email: 'bob@test.dev',
      password: 'secret123'
    }).expect(201);
    ctx.bobToken = u2.body.data.token;
  });

  await t.test('duplicate email is rejected', async () => {
    await api().post('/api/auth/register').send({
      name: 'A2',
      email: 'alice@test.dev',
      password: 'secret123'
    }).expect(409);
  });

  await t.test('login: wrong password 401, right password 200, /me works', async () => {
    await api().post('/api/auth/login').send({
      email: 'alice@test.dev',
      password: 'wrongpass'
    }).expect(401);
    await api().post('/api/auth/login').send({
      email: 'alice@test.dev',
      password: 'secret123'
    }).expect(200);
    const me = await api().get('/api/auth/me').set(auth(ctx.aliceToken)).expect(200);
    assert.equal(me.body.data.name, 'Alice');
    await api().get('/api/auth/me').expect(401);
    await api().get('/api/auth/me').set(auth('garbage')).expect(401);
  });

  await t.test('Firebase login returns 503 when Firebase is not configured', async () => {

```

```

    await api().post('/api/auth/firebase').send({ idToken: 'abc' }).expect(503);
  });

  await t.test('POST /api/businesses: plain user is forbidden', async () => {
    await api().post('/api/businesses').set(auth(ctx.aliceToken))
      .send({ name: 'Nope', category: 'cafe', address: { city: 'Mumbai' } }).expect(403);
  });

  await t.test('POST /api/businesses: validation errors', async () => {
    const res = await api().post('/api/businesses').set(auth(ctx.ownerToken))
      .send({ name: 'Bad', latitude: 19 }).expect(422);
    const fields = res.body.errors.map((e) => e.field);
    assert.ok(fields.includes('category'));
    assert.ok(fields.includes('address.city'));
    assert.ok(fields.includes('longitude'));
  });

  await t.test('POST /api/businesses: unknown category 400', async () => {
    await api().post('/api/businesses').set(auth(ctx.ownerToken))
      .send({ name: 'X', category: 'spaceship', address: { city: 'Mumbai' } }).expect(400);
  });

  await t.test('POST /api/businesses multipart with photo', async () => {
    const res = await api().post('/api/businesses').set(auth(ctx.ownerToken))
      .field('name', 'Bean There Cafe')
      .field('description', 'Great pour-over coffee')
      .field('category', 'cafe')
      .field('tags', 'coffee, wifi')
      .field('address[city]', 'Mumbai')
      .field('address[street]', '12 Hill Road')
      .field('latitude', '19.0544')
      .field('longitude', '72.8406')
      .attach('photos', PNG, { filename: 'front.png', contentType: 'image/png' })
      .expect(201);
    const b = res.body.data;
    ctx.businessId = b._id;
    assert.equal(b.category.slug, 'cafe');
    assert.deepEqual(b.tags, ['coffee', 'wifi']);
    assert.equal(b.address.city, 'Mumbai');
    assert.deepEqual(b.location.coordinates, [72.8406, 19.0544]);
    assert.equal(b.photos.length, 1);
    assert.equal(b.photos[0].provider, 'local');
    ctx.photoId = b.photos[0]._id;

    // the stored file is served back
    const url = new URL(b.photos[0].url);
    const img = await api().get(url.pathname).expect(200);
    assert.equal(img.headers['content-type'], 'image/png');
  });

  await t.test('upload rejects non-image files', async () => {
    await api().post(`/api/businesses/${ctx.businessId}/photos`).set(auth(ctx.ownerToken))
      .attach('photos', Buffer.from('hello'), { filename: 'x.txt', contentType:
'text/plain' }).expect(400);
  });

  await t.test('POST /api/businesses JSON (second and third business)', async () => {
    const gym = await api().post('/api/businesses').set(auth(ctx.ownerToken)).send({
      name: 'Iron Temple', category: 'Gym', address: { city: 'Pune' }, latitude: 18.5196, longitude:
73.8553,
    }).expect(201);
    ctx.gymId = gym.body.data._id;

    await api().post('/api/businesses').set(auth(ctx.ownerToken)).send({
      name: 'Chai Point', description: 'Tea and snacks', category: 'cafe', tags: ['tea'],
      address: { city: 'Mumbai' }, latitude: 19.0728, longitude: 72.8347,
    }).expect(201);
  });

  await t.test('GET /api/businesses lists with filters and pagination', async () => {
    const all = await api().get('/api/businesses').expect(200);
    assert.equal(all.body.pagination.total, 3);

    const cafes = await api().get('/api/businesses?category=cafe').expect(200);
    assert.equal(cafes.body.data.length, 2);

    const pune = await api().get('/api/businesses?city=pune').expect(200);
    assert.equal(pune.body.data.length, 1);
    assert.equal(pune.body.data[0].name, 'Iron Temple');
  });

```

```

const paged = await api().get('/api/businesses?limit=2&page=2').expect(200);
assert.equal(paged.body.data.length, 1);
assert.equal(paged.body.pagination.pages, 2);

const unknown = await api().get('/api/businesses?category=nothing').expect(200);
assert.equal(unknown.body.data.length, 0);

const mine = await api().get('/api/businesses?owner=me').set(auth(ctx.ownerToken)).expect(200);
assert.equal(mine.body.data.length, 3);
await api().get('/api/businesses?owner=me').expect(401);
});

await t.test('GET /api/businesses/:id', async () => {
  const res = await api().get(`/api/businesses/${ctx.businessId}`).expect(200);
  assert.equal(res.body.data.name, 'Bean There Cafe');
  assert.equal(res.body.data.owner.name, 'Owner');
  await api().get('/api/businesses/not-an-id').expect(422);
  await api().get(`/api/businesses/${new mongoose.Types.ObjectId()}`).expect(404);
});

await t.test('GET /api/businesses/search?keyword=cafe', async () => {
  // "cafe" matches the category, so Chai Point is found even though its text doesn't say cafe
  const res = await api().get('/api/businesses/search?keyword=cafe').expect(200);
  const names = res.body.data.map((b) => b.name).sort();
  assert.deepEqual(names, ['Bean There Cafe', 'Chai Point']);

  const partial = await api().get('/api/businesses/search?keyword=pour').expect(200);
  assert.equal(partial.body.data.length, 1);

  const byTag = await api().get('/api/businesses/search?keyword=wifi').expect(200);
  assert.equal(byTag.body.data[0].name, 'Bean There Cafe');

  const regexSafe = await api().get('/api/businesses/search?keyword=.*').expect(200);
  assert.equal(regexSafe.body.data.length, 0);

  await api().get('/api/businesses/search').expect(422);
});

await t.test('GET /api/businesses/near returns nearest first with distance', async () => {
  // Bandra, Mumbai
  const res = await api().get('/api/businesses/near?lat=19.0596&lng=72.8295&radius=10').expect(200);
  assert.equal(res.body.data.length, 2);
  assert.equal(res.body.data[0].name, 'Bean There Cafe');
  assert.ok(res.body.data[0].distanceKm < res.body.data[1].distanceKm);
  assert.equal(res.body.data[0].category.slug, 'cafe');

  const wide = await api().get('/api/businesses/near?lat=19.0596&lng=72.8295&radius=100').expect(200);
  assert.equal(wide.body.data.length, 2); // Pune is ~120km away

  const gymsOnly = await api().get('/api/businesses/near?lat=18.52&lng=73.85&category=gym').expect(200);
  assert.equal(gymsOnly.body.data.length, 1);

  await api().get('/api/businesses/near?lat=200&lng=72').expect(422);
  await api().get('/api/businesses/near').expect(422);
});

await t.test('PUT /api/businesses/:id: owner can update, others cannot', async () => {
  await api().put(`/api/businesses/${ctx.businessId}`).set(auth(ctx.aliceToken)).send({ name: 'Hacked' }).expect(403);

  // a second owner who doesn't own this business
  const other = await api().post('/api/auth/register')
    .send({ name: 'Other', email: 'other@test.dev', password: 'secret123', role: 'owner' }).expect(201);
  ctx.otherOwnerToken = other.body.data.token;
  await api().put(`/api/businesses/${ctx.businessId}`).set(auth(ctx.otherOwnerToken)).send({ name: 'Hacked' }).expect(403);

  await api().put(`/api/businesses/${ctx.businessId}`).set(auth(ctx.ownerToken)).send({ averageRating: 5 }).expect(422);

  const res = await api().put(`/api/businesses/${ctx.businessId}`).set(auth(ctx.ownerToken))
    .send({ phone: '+91 98200 11111', address: { zip: '400050' } }).expect(200);
  assert.equal(res.body.data.phone, '+91 98200 11111');
  assert.equal(res.body.data.address.zip, '400050');
  assert.equal(res.body.data.address.city, 'Mumbai'); // partial address update keeps other fields

```

```

    assert.equal(res.body.data.name, 'Bean There Cafe');
  });

  await t.test('POST /api/reviews and rating aggregation', async () => {
    const r1 = await api().post('/api/reviews').set(auth(ctx.aliceToken))
      .send({ business: ctx.businessId, rating: 5, title: 'Loved it', comment: 'Great
coffee' }).expect(201);
    ctx.aliceReviewId = r1.body.data._id;
    assert.equal(r1.body.data.user.name, 'Alice');
    assert.deepEqual(r1.body.businessRating, { averageRating: 5, reviewCount: 1 });

    const r2 = await api().post('/api/reviews').set(auth(ctx.bobToken))
      .send({ business: ctx.businessId, rating: 2, comment: 'Too crowded' }).expect(201);
    ctx.bobReviewId = r2.body.data._id;
    assert.deepEqual(r2.body.businessRating, { averageRating: 3.5, reviewCount: 2 });

    const b = await api().get(`/api/businesses/${ctx.businessId}`).expect(200);
    assert.equal(b.body.data.averageRating, 3.5);
    assert.equal(b.body.data.reviewCount, 2);
  });

  await t.test('review rules: duplicate 409, own business 403, bad rating 422, auth 401', async () => {
    await api().post('/api/reviews').set(auth(ctx.aliceToken)).send({ business: ctx.businessId, rating:
4 }).expect(409);
    await api().post('/api/reviews').set(auth(ctx.ownerToken)).send({ business: ctx.businessId, rating:
5 }).expect(403);
    await api().post('/api/reviews').set(auth(ctx.aliceToken)).send({ business: ctx.gymId, rating:
6 }).expect(422);
    await api().post('/api/reviews').set(auth(ctx.aliceToken)).send({ business: ctx.gymId, rating:
4.5 }).expect(422);
    await api().post('/api/reviews').send({ business: ctx.gymId, rating: 4 }).expect(401);
    await api().post('/api/reviews').set(auth(ctx.aliceToken))
      .send({ business: new mongoose.Types.ObjectId().toString(), rating: 4 }).expect(404);
  });

  await t.test('GET /api/reviews/business/:id with breakdown and sorting', async () => {
    const res = await api().get(`/api/reviews/business/${ctx.businessId}?sort=lowest`).expect(200);
    assert.equal(res.body.data.length, 2);
    assert.equal(res.body.data[0].rating, 2);
    assert.deepEqual(res.body.summary.breakdown, { 1: 0, 2: 1, 3: 0, 4: 0, 5: 1 });
    assert.equal(res.body.summary.averageRating, 3.5);
    await api().get(`/api/reviews/business/${new mongoose.Types.ObjectId()}`).expect(404);
  });

  await t.test('PUT /api/reviews/:id only by author, re-aggregates', async () => {
    await api().put(`/api/reviews/${ctx.bobReviewId}`).set(auth(ctx.aliceToken)).send({ rating:
5 }).expect(403);
    const res = await api().put(`/api/reviews/${ctx.bobReviewId}`).set(auth(ctx.bobToken)).send({ rating: 4
}).expect(200);
    assert.equal(res.body.data.rating, 4);
    assert.deepEqual(res.body.businessRating, { averageRating: 4.5, reviewCount: 2 });
  });

  await t.test('owner response to review', async () => {
    const url = `/api/reviews/${ctx.bobReviewId}/response`;
    await api().post(url).set(auth(ctx.bobToken)).send({ text: 'x' }).expect(403); // not an owner
    await api().post(url).set(auth(ctx.otherOwnerToken)).send({ text: 'x' }).expect(403); // not this
business
    await api().post(url).set(auth(ctx.ownerToken)).send({ text: '' }).expect(422);

    const res = await api().post(url).set(auth(ctx.ownerToken)).send({ text: 'Thanks, we added
seating!' }).expect(200);
    assert.equal(res.body.data.ownerResponse.text, 'Thanks, we added seating!');
    assert.ok(res.body.data.ownerResponse.respondedAt);

    const edited = await api().put(url).set(auth(ctx.ownerToken)).send({ text: 'Updated
reply' }).expect(200);
    assert.equal(edited.body.data.ownerResponse.text, 'Updated reply');

    const list = await api().get(`/api/reviews/business/${ctx.businessId}`).expect(200);
    assert.equal(list.body.data.find((r) => r._id === ctx.bobReviewId).ownerResponse.text, 'Updated
reply');

    await api().delete(url).set(auth(ctx.ownerToken)).expect(200);
    await api().delete(url).set(auth(ctx.ownerToken)).expect(404);
  });

  await t.test('GET /api/reviews/me', async () => {
    const res = await api().get('/api/reviews/me').set(auth(ctx.aliceToken)).expect(200);
  });

```

```

    assert.equal(res.body.data.length, 1);
    assert.equal(res.body.data[0].business.name, 'Bean There Cafe');
  });

  await t.test('search can filter by minRating and sorts by rating', async () => {
    const res = await api().get(`/api/businesses/search?keyword=cafe&minRating=4`).expect(200);
    assert.equal(res.body.data.length, 1);
    assert.equal(res.body.data[0].name, 'Bean There Cafe');
  });

  await t.test('DELETE /api/reviews/:id re-aggregates', async () => {
    await api().delete(`/api/reviews/${ctx.aliceReviewId}`).set(auth(ctx.bobToken)).expect(403);
    const res = await api().delete(`/api/reviews/${ctx.aliceReviewId}`).set(auth(ctx.aliceToken)).expect(200);
    assert.deepEqual(res.body.businessRating, { averageRating: 4, reviewCount: 1 });
  });

  await t.test('serverless deploy without Firebase: uploads fail with 503, no local writes', async () => {
    const env = require('../src/config/env');
    env.isServerless = true;
    try {
      const res = await api().post(`/api/businesses/${ctx.businessId}/photos`).set(auth(ctx.ownerToken))
        .attach('photos', PNG, { filename: 'a.png', contentType: 'image/png' })
        .expect(503);
      assert.match(res.body.message, /Firebase Storage/);
      const health = await api().get('/api/health').expect(200);
      assert.equal(health.body.photoStorage, 'unavailable');
      // JSON-only creates still work
      await api().post('/api/businesses').set(auth(ctx.ownerToken))
        .send({ name: 'No Photo Cafe', category: 'cafe', address: { city: 'Delhi' } }).expect(201);
    } finally {
      env.isServerless = false;
    }
  });

  await t.test('photos: add and delete', async () => {
    const added = await api().post(`/api/businesses/${ctx.businessId}/photos`).set(auth(ctx.ownerToken))
      .attach('photos', PNG, { filename: 'a.png', contentType: 'image/png' })
      .attach('photos', PNG, { filename: 'b.png', contentType: 'image/png' })
      .expect(201);
    assert.equal(added.body.data.length, 3);

    await api().post(`/api/businesses/${ctx.businessId}/photos`).set(auth(ctx.ownerToken)).expect(400);
    await api().delete(`/api/businesses/${ctx.businessId}/photos/${ctx.photoId}`).set(auth(ctx.aliceToken)).expect(403);

    const photo = added.body.data.find((p) => p._id === ctx.photoId);
    const file = path.join(LOCAL_DIR, photo.path);
    assert.ok(fs.existsSync(file));
    const res = await api().delete(`/api/businesses/${ctx.businessId}/photos/${ctx.photoId}`).set(auth(ctx.ownerToken)).expect(200);
    assert.equal(res.body.data.length, 2);
    assert.ok(!fs.existsSync(file));
  });

  await t.test('categories: list with counts, admin-only writes', async () => {
    const res = await api().get('/api/categories').expect(200);
    const cafe = res.body.data.find((c) => c.slug === 'cafe');
    assert.equal(cafe.businessCount, 3);

    await api().post('/api/categories').set(auth(ctx.ownerToken)).send({ name: 'Bakery' }).expect(403);
    const created = await api().post('/api/categories').set(auth(ctx.adminToken)).send({ name: 'Book Store' }).expect(201);
    assert.equal(created.body.data.slug, 'book-store');
    await api().post('/api/categories').set(auth(ctx.adminToken)).send({ name: 'Book Store' }).expect(409);

    await api().delete(`/api/categories/${cafe._id}`).set(auth(ctx.adminToken)).expect(409); // in use
    await api().delete(`/api/categories/${created.body.data._id}`).set(auth(ctx.adminToken)).expect(200);
  });

  await t.test('DELETE /api/businesses/:id removes reviews too', async () => {
    await api().delete(`/api/businesses/${ctx.businessId}`).set(auth(ctx.aliceToken)).expect(403);
    await api().delete(`/api/businesses/${ctx.businessId}`).set(auth(ctx.ownerToken)).expect(200);
    await api().get(`/api/businesses/${ctx.businessId}`).expect(404);
    const Review = mongoose.model('Review');
    assert.equal(await Review.countDocuments({ business: ctx.businessId }), 0);
  });

  await t.test('admin can manage any business', async () => {

```

```

    await api().put(`/api/businesses/${ctx.gymId}`).set(auth(ctx.adminToken)).send({ description:
'Moderated' }).expect(200);
  });

  await t.test('Swagger UI page and spec', async () => {
    const page = await api().get('/api-docs').expect(200);
    assert.match(page.headers['content-type'], /html/);
    assert.match(page.text, /swagger-ui-bundle\.js/);
    assert.match(page.headers['content-security-policy'], /cdn\.jsdelivr\.net/);
    const init = await api().get('/api-docs/init.js').expect(200);
    assert.match(init.headers['content-type'], /javascript/);
    const spec = await api().get('/api-docs.json').expect(200);
    assert.equal(spec.body.openapi, '3.0.3');
  });

  await t.test('unknown route and malformed JSON', async () => {
    await api().get('/api/nope').expect(404);
    const res = await api().post('/api/auth/login').set('Content-Type',
'application/json').send({'bad'}).expect(400);
    assert.equal(res.body.message, 'Malformed JSON body');
  });
});

```